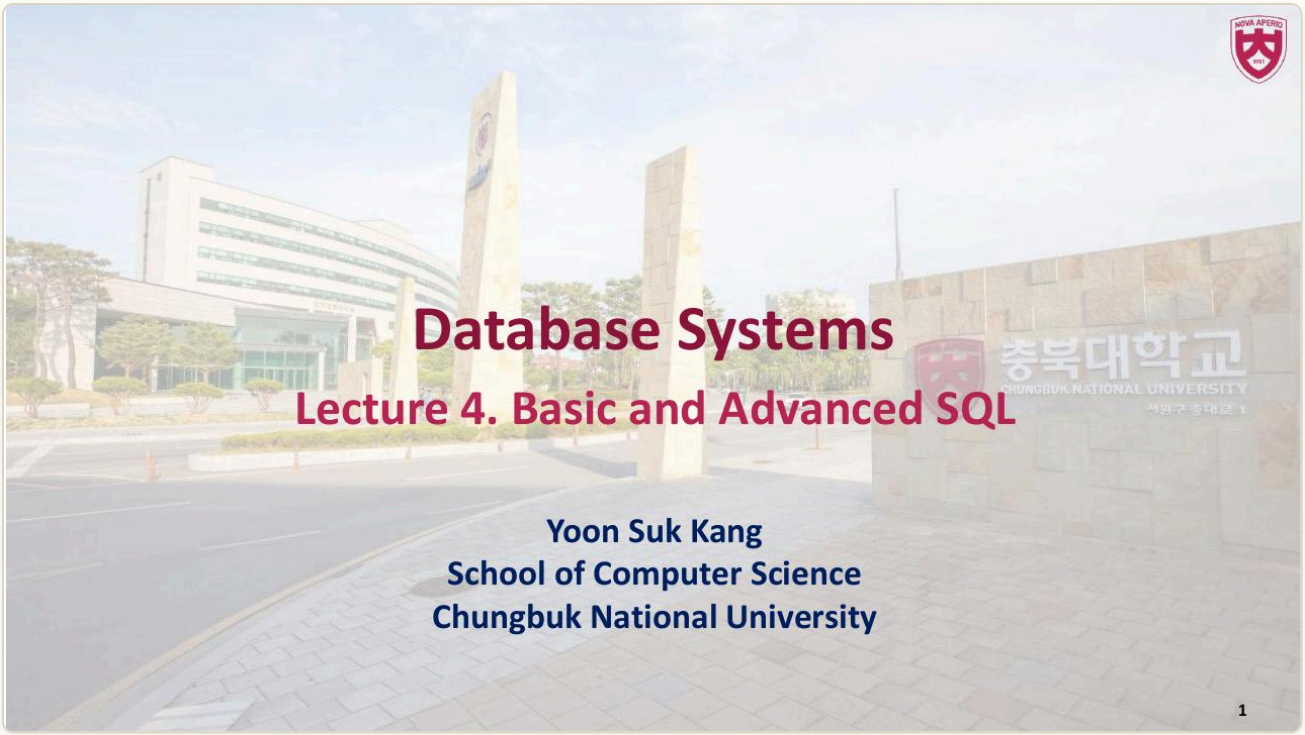
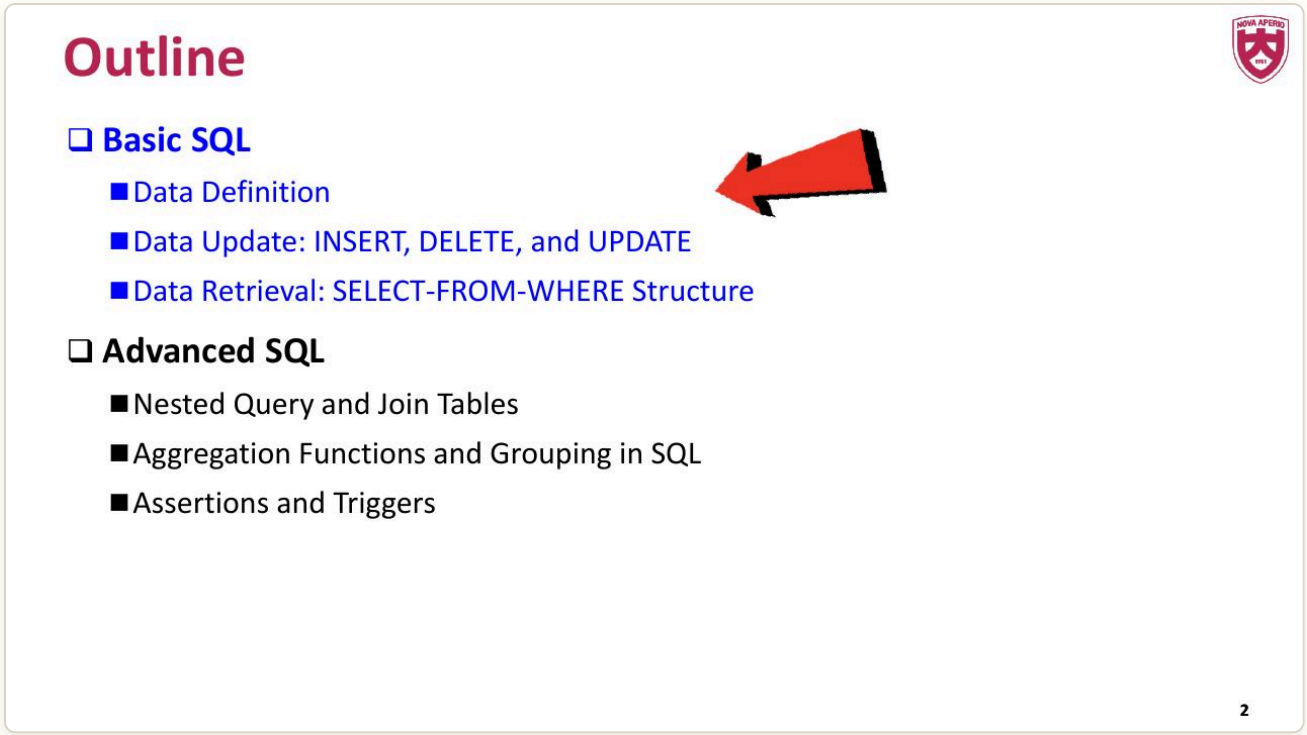


데이터베이스 시스템 — 4강: 기본·고급 SQL

- Yoon Suk Kang · 충북대 소프트웨어학부

필기 ✍️

💡 이해 시험에 SQL을 직접 쓰는 문제가 나올 가능성 큼 — 연습 2~5를 손으로 써 보기

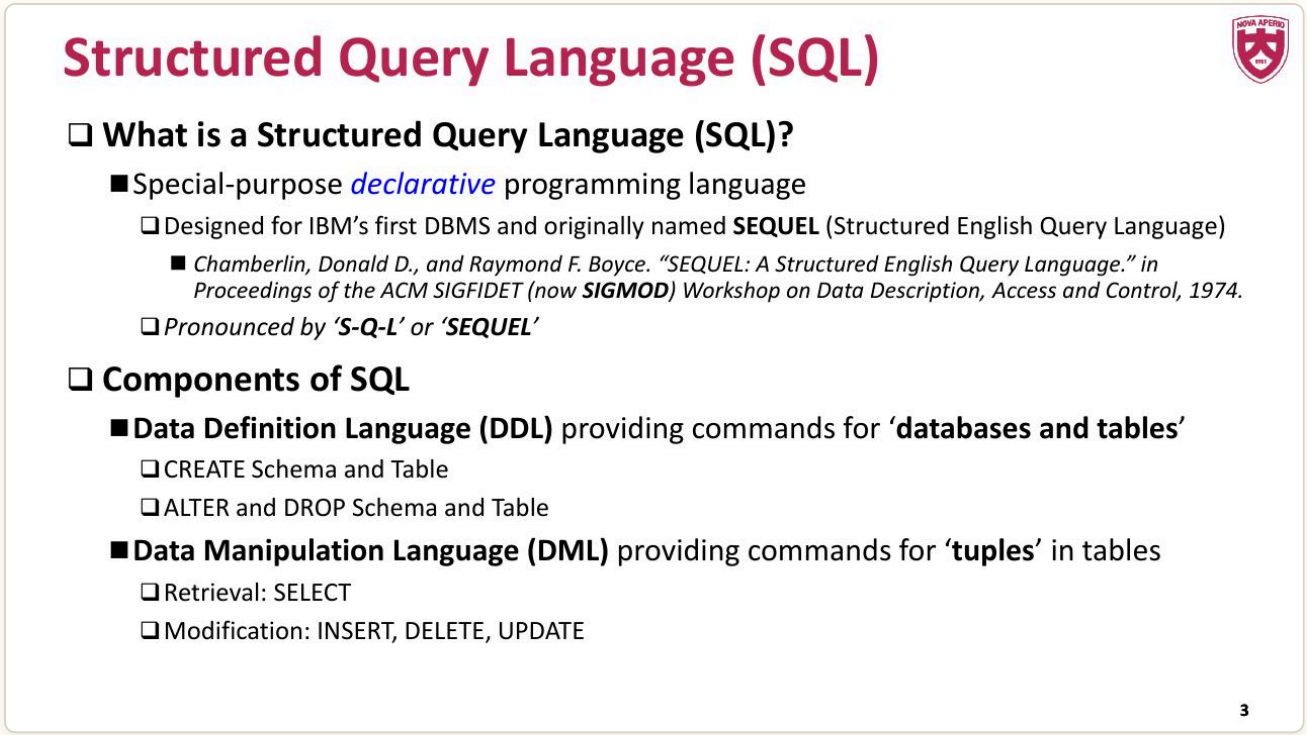


목차 (Outline)

- **Basic SQL** — 데이터 정의 / 갱신: INSERT·DELETE·UPDATE / 검색: SELECT-FROM-WHERE
- **Advanced SQL** — 중첩 질의와 조인 테이블 / 집계 함수와 그룹화 / 어서션과 트리거

필기 ✍️

— 넘어가도 되는 쪽



Structured Query Language (SQL)

❑ What is a Structured Query Language (SQL)?

- ■ Special-purpose *declarative* programming language
  - ❑ Designed for IBM's first DBMS and originally named **SEQUEL** (Structured English Query Language)
    - ■ *Chamberlin, Donald D., and Raymond F. Boyce. "SEQUEL: A Structured English Query Language." in Proceedings of the ACM SIGFIDET (now **SIGMOD**) Workshop on Data Description, Access and Control, 1974.*
  - ❑ *Pronounced by '**S-Q-L**' or '**SEQUEL**'*

❑ Components of SQL

- ■ **Data Definition Language (DDL)** providing commands for '**databases and tables**'
  - ❑ CREATE Schema and Table
  - ❑ ALTER and DROP Schema and Table
- ■ **Data Manipulation Language (DML)** providing commands for '**tuples**' in tables
  - ❑ Retrieval: SELECT
  - ❑ Modification: INSERT, DELETE, UPDATE

3

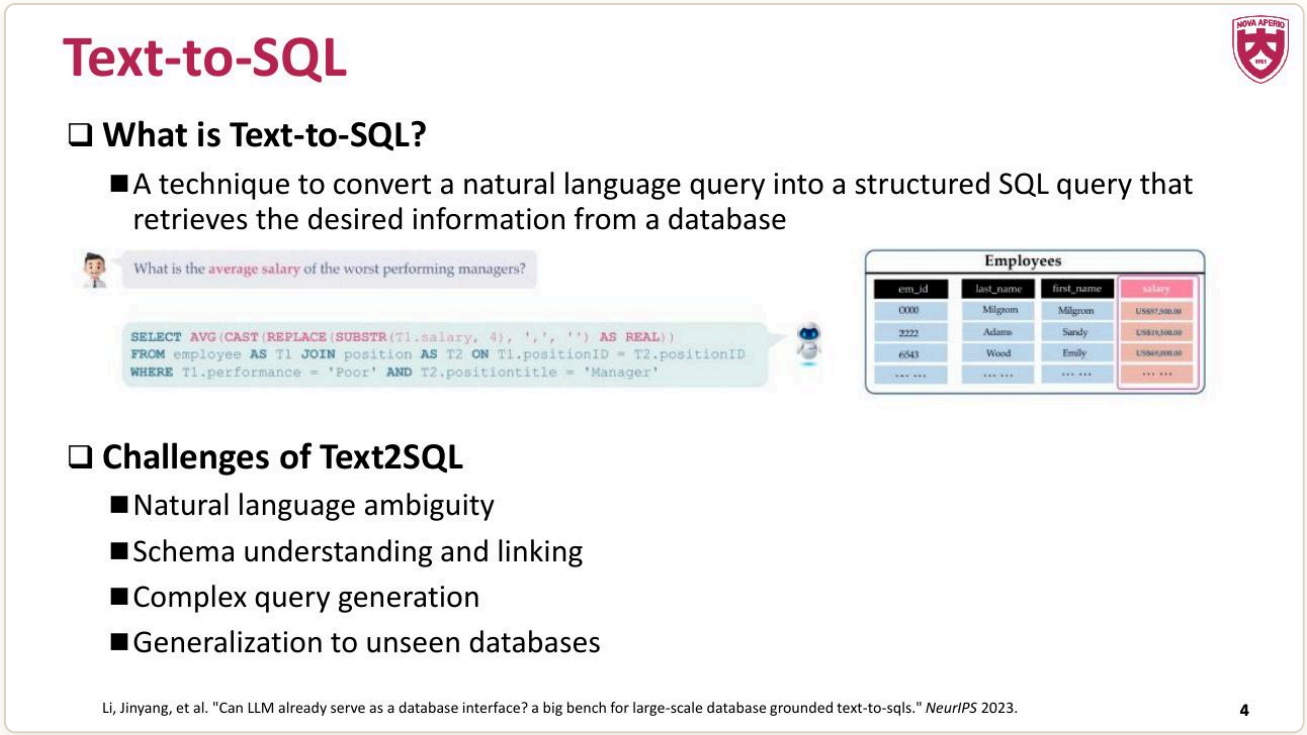
SQL이란?

- 특수 목적 **선언형(declarative)** 프로그래밍 언어
  - IBM 최초 DBMS용으로 설계, 원래 이름 **SEQUEL**(Structured English Query Language)
    - Chamberlin & Boyce, 1974
  - 'S-Q-L' 또는 '시퀄'로 읽음
- SQL의 구성
  - **DDL**(Data Definition Language): DB·테이블에 대한 명령 — CREATE / ALTER / DROP (스키마·테이블)
  - **DML**(Data Manipulation Language): 테이블의 튜플에 대한 명령 — 검색 SELECT / 수정 INSERT·DELETE·UPDATE

필기 ✍️

★ 시험 DDL = CREATE·ALTER·DROP / DML = SELECT·INSERT·DELETE·UPDATE — 분류 문제 단골

⚠ 주의 SELECT도 DML이다 (검색도 조작에 포함)



Text-to-SQL

- 자연어 질문을 원하는 정보를 가져오는 SQL로 바꾸는 기법
- 어려운 점
  - 자연어의 **모호성**
  - **스키마 이해와 연결(linking)**
  - **복잡한 질의** 생성
  - 처음 보는 DB로의 **일반화**

필기 ✍️

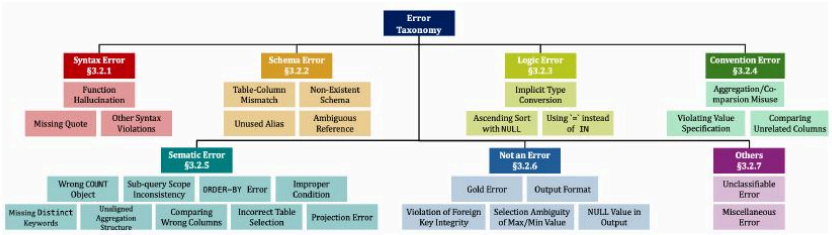
💡 이해 LLM 시대에도 SQL을 알아야 하는 이유를 말하는 도입 슬라이드

Limitations of Text-to-SQL

❑ Recent LLM-based Text-to-SQL Performance on a Benchmark

	Model	Code	Size	Self Consistency	Dev (%)	Test (%)
	Human Performance *Data Engineers + DB Students*					92.96
1 Mar 12, 2026	Gemini-SQL (Multitask SFT + Gemini-2.5-Pro) *Google Cloud Databases* Google Cloud Databases		UNK	*Few*	72.62	76.63
2 Dec 4, 2025	Q-SQL *AWS-Quick Science* [Ravi Shankar et al.'25]		30B-3B-MoE	*Many*	72.99	75.47
3 July 06, 2025	Databricks RLVR 32B *Databricks Mosaic Research* Databricks Mosaic Research		32B	*Few*	-	75.68
4 Dec 19, 2025	SiriusAI-Text2SQL-32B-v2 *Tencent Data & Computation Platform Department*		32B	*Many*	72.82	75.01

	Model	Code	Size	Oracle Knowledge	Dev (%)	Test (%)
	Human Performance *Data Engineers + DB Students*			✓		92.96
1 Dec 10, 2025	AskData + GPT-4o *AT&T CDO - DSAIR* [Shkapenyuk et al. '25]		UNK	✓	77.64	81.95
2 Sep 21, 2025	Agentar-Scale-SQL *Ant Group* [Pengfei Wang et al. '25]	[link]	UNK	✓	74.90	81.67
3 July 14, 2025	LongData-SQL *LongShine AI Research*		UNK	✓	74.32	77.53
4 Jan 22, 2026	Zhiwen-Lingsi-Agent *China Telecom, TeleAI*		UNK	✓	73.53	76.63
5 Jan 29, 2026	DeepEye-SQL *Anonymous*		UNK	✓	73.53	76.58



Many types of errors

Recent LLM-based Text-to-SQL technologies are powerful, but they are far from perfect
Thus, a solid understanding of SQL is crucial!

Li, Jinyang, et al. "Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-sqls." *NeurIPS* 2023.
Shen, Jiawei, et al. "A study of in-context-learning-based text-to-sql errors." *arXiv preprint arXiv:2501.09310* (2025).

5

Text-to-SQL의 한계

- 최근 LLM 기반 Text-to-SQL의 벤치마크 성능 — 오류 유형이 많다
  - 강력하지만 **완벽과는 거리가 멀다**
  - → 그래서 **SQL을 제대로 이해하는 것이 중요!**

필기 ✍️

— 넘어가도 되는 쪽

Creating a Schema in SQL

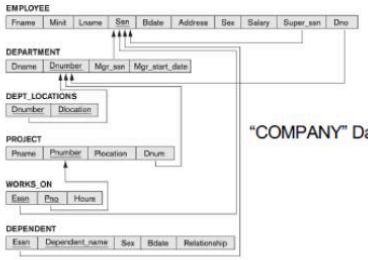
❑ **What is a Schema in SQL?**

- ■ Tables, types, constraints, views, domains, and others that describe the schema
  - ❑ Identified by a **schema name**
- ■ Authorization identifier to indicate the user who owns the schema

❑ **CREATE SCHEMA Statement**

- ■ Specify a schema name and an authorization identifier

CREATE SCHEMA COMPANY **AUTHORIZATION** 'Jsmith';



"COMPANY" Database

6

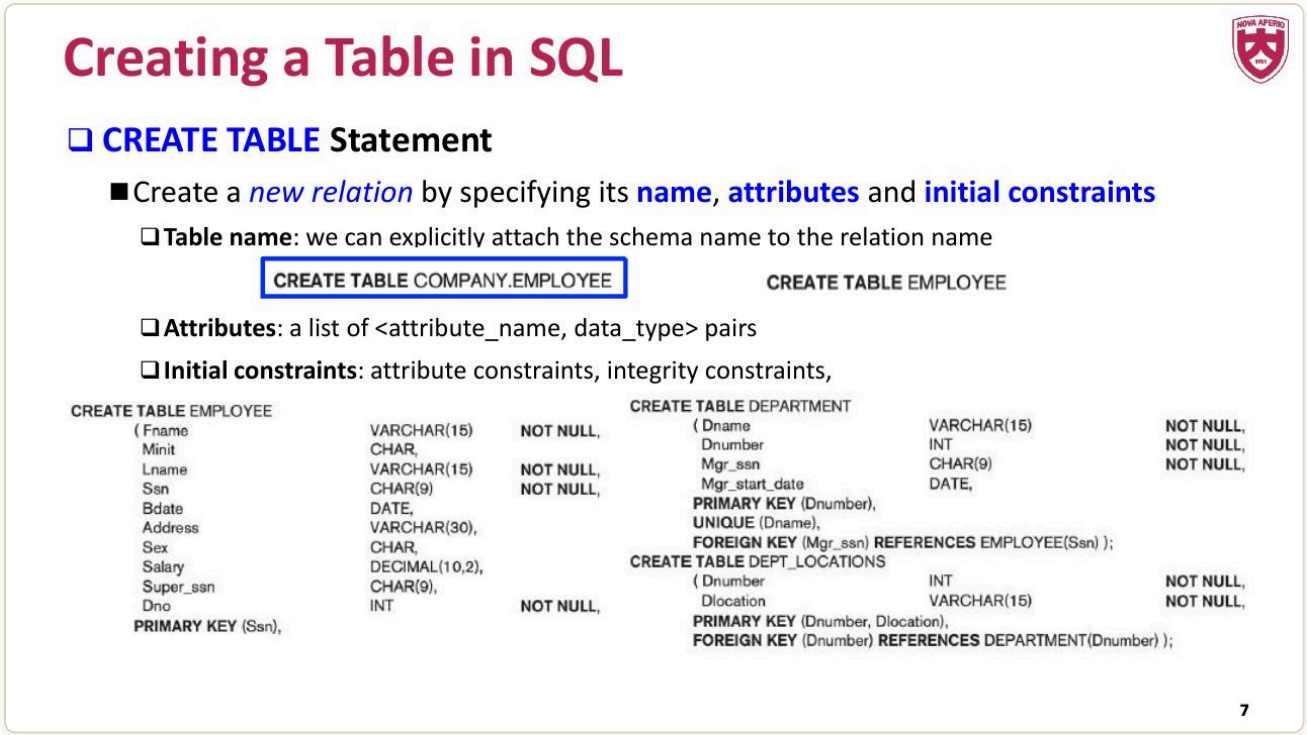
스키마 만들기

- SQL의 **스키마**: 스키마를 기술하는 테이블·타입·제약·뷰·도메인 등
  - **스키마 이름**으로 식별
  - 스키마 주인을 나타내는 **authorization identifier**
- CREATE SCHEMA 문 — 스키마 이름과 권한 식별자 지정

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

필기 ✍️

— 넘어가도 되는 쪽



테이블 만들기

- **CREATE TABLE**: 이름·속성·초기 제약을 정해 새 릴레이션 생성
  - 테이블 이름 앞에 스키마 이름을 붙일 수 있음 (COMPANY.EMPLOYEE)
  - 속성: <속성 이름, 데이터 타입> 쌍의 목록
  - 초기 제약: 속성 제약, 무결성 제약

```
CREATE TABLE EMPLOYEE ( Fname VARCHAR(15) NOT NULL, …, Ssn CHAR(9) NOT NULL, …, Dno INT NOT NULL, PRIMARY KEY (Ssn) );
```

필기 ✍️

💡 이해 그림은 COMPANY 6개 표의 CREATE 문 — 자료형(VARCHAR, CHAR, INT, DATE, DECIMAL) 눈에 익히기

Specifying Attribute Constraints

❑ NOT NULL Constraint for Attributes

- This constraint is always implicitly specified for the primary key attributes
  - i.e., *Entity integrity constraint*

❑ Default Values for Attributes

- It is possible to define a default value for an attribute by appending the clause **DEFAULT** <value> to an attribute definition
  - The default value is included in any new tuple if *an explicit value is not provided* for that attribute

```
CREATE TABLE EMPLOYEE
    ( Fname          VARCHAR(15)     NOT NULL,
      Minit          CHAR,
      Lname          VARCHAR(15)     NOT NULL,
      Ssn            CHAR(9)         NOT NULL,
      Bdate          DATE,
      Address        VARCHAR(30),
      Sex            CHAR,
      Salary         DECIMAL(10,2),
      Super_ssn      CHAR(9),
      Dno            INT             NOT NULL,
    PRIMARY KEY (Ssn),
```

```
CREATE TABLE DEPARTMENT
    ( ... ,
      Mgr_ssn CHAR(9)        NOT NULL      DEFAULT '888665555',
      ... ,
    CONSTRAINT DEPTPK
      PRIMARY KEY(Dnumber),
    CONSTRAINT DEPTSK
      UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
      FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
                  ON DELETE SET DEFAULT     ON UPDATE CASCADE);
```

Specifying a default department for a new employee

8

속성 제약 지정

- **NOT NULL**
  - 기본키 속성에는 **항상 암묵적으로** 지정됨 — 즉 **개체 무결성**
- **DEFAULT <값>**
  - 속성 정의 뒤에 붙여 기본값 지정
  - 새 튜플에 그 속성 값을 안 주면 기본값이 들어감

```
Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555'
```

필기 ✍️

★ 시험 PK에는 NOT NULL이 자동 = entity integrity

Specifying Key Constraints



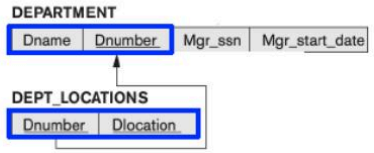
❑ PRIMARY KEY Clause

■ Specifies one or more attributes that make up the primary key of a relation

❑ UNIQUE Clause

■ Specifies alternate (unique) keys (i.e., *candidate keys*)

```
CREATE TABLE DEPARTMENT
        ( Dname                 VARCHAR(15)         NOT NULL,
          Dnumber               INT                 NOT NULL,
          Mgr_ssn               CHAR(9)             NOT NULL,
          Mgr_start_date        DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber               INT                 NOT NULL,
          Dlocation             VARCHAR(15)         NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```



9

키 제약 지정

- **PRIMARY KEY** 절 — 기본키를 이루는 속성(들)
- **UNIQUE** 절 — 대체(unique) 키 = **후보키**

```
PRIMARY KEY (Dnumber), UNIQUE (Dname)
```

  - CONSTRAINT 이름을 붙일 수도 있음 (DEPTPK, DEPTSK ...)

필기 ✍️

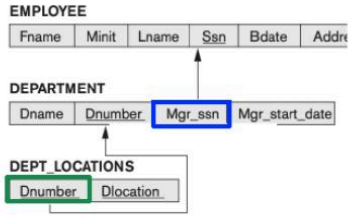
★ 시험 UNIQUE = candidate(alternate) key

Specifying Referential Integrity Constraints

❑ FOREIGN KEY Clause

■ Specifies a foreign key that refers to a primary key of another relation

```
CREATE TABLE DEPARTMENT
        ( Dname                 VARCHAR(15)     NOT NULL,
          Dnumber               INT             NOT NULL,
          Mgr_ssn               CHAR(9)         NOT NULL,
          Mgr_start_date        DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber               INT             NOT NULL,
          Dlocation             VARCHAR(15)     NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```



Referential integrity constraint can be violated
when the primary key attribute values are updated or deleted.

The default action of SQL is to ***reject*** the operation
Q. Any other way to address this issue? → **Referential Triggered Actions**

10

참조 무결성 제약 지정

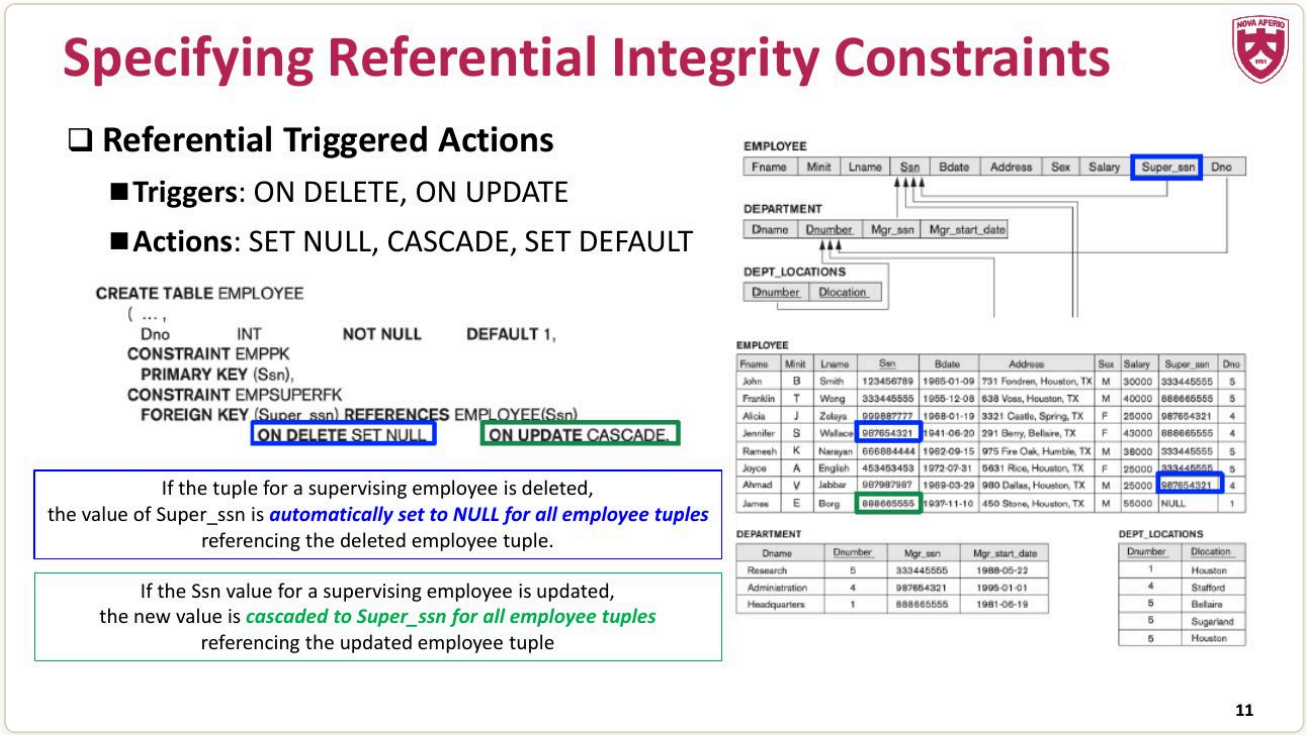
- **FOREIGN KEY** 절 — 다른 릴레이션의 기본키를 참조하는 외래키

```
FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
```

- PK 값이 **수정·삭제**되면 참조 무결성이 깨질 수 있다
  - SQL의 기본 동작은 그 연산을 **거부(reject)**
  - Q. 다른 방법은? → **Referential Triggered Actions**

필기 ✍️

★ 시험 기본 = reject (RESTRICT)



Specifying Referential Integrity Constraints

❑ **Referential Triggered Actions**

■ **Triggers**: ON DELETE, ON UPDATE

■ **Actions**: SET NULL, CASCADE, SET DEFAULT

```
CREATE TABLE EMPLOYEE
   ( ... ,
    Dno        INT          NOT NULL      DEFAULT 1,
  CONSTRAINT EMPPK
    PRIMARY KEY (Ssn),
  CONSTRAINT EMPSUPERFK
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
                ON DELETE SET NULL        ON UPDATE CASCADE,
```

If the tuple for a supervising employee is deleted,
the value of Super_ssn is ***automatically set to NULL for all employee tuples***
referencing the deleted employee tuple.

If the Ssn value for a supervising employee is updated,
the new value is ***cascaded to Super_ssn for all employee tuples***
referencing the updated employee tuple

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

11

참조 트리거 동작 ①

- 트리거: **ON DELETE, ON UPDATE**
- 동작: **SET NULL, CASCADE, SET DEFAULT**

```
CONSTRAINT EMPSUPERFK FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn) ON DELETE SET NULL
ON UPDATE CASCADE
```

- 감독자 튜플을 지우면 → 그를 참조하던 모든 직원의 Super_ssn이 자동으로 **NULL**
- 감독자의 Ssn을 바꾸면 → 새 값이 참조하는 모든 직원의 Super_ssn으로 **연쇄(cascade)**

필기 ✍️

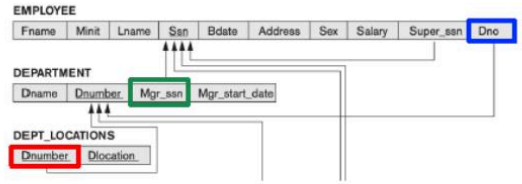
★ 시험 동작 3개 뜻: SET NULL(비우기) · CASCADE(같이 지우기/같이 바꾸기) · SET DEFAULT(기본값으로)

Specifying Referential Integrity Constraints

❑ **Referential Triggered Actions**

- **Triggers**: ON DELETE, ON UPDATE
- **Actions**: SET NULL, CASCADE, SET DEFAULT

```
CREATE TABLE EMPLOYEE
    ( ... ,
     FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)
                ON DELETE SET DEFAULT     ON UPDATE CASCADE);
CREATE TABLE DEPARTMENT
    ( ... ,
     Mgr_ssn CHAR(9)          NOT NULL        DEFAULT '888665555',
     ... ,
    CONSTRAINT DEPTPK
     PRIMARY KEY(Dnumber),
    CONSTRAINT DEPTSK
     UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
     FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
                ON DELETE SET DEFAULT     ON UPDATE CASCADE);
CREATE TABLE DEPT_LOCATIONS
    ( ... ,
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
                ON DELETE CASCADE         ON UPDATE CASCADE);
```



EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

12

참조 트리거 동작 ②

- 예: EMPLOYEE.Dno — ON DELETE SET DEFAULT ON UPDATE CASCADE (Dno DEFAULT 1)
- 예: DEPARTMENT.Mgr_ssn — ON DELETE SET DEFAULT ON UPDATE CASCADE (DEFAULT '888665555')
- 예: DEPT_LOCATIONS.Dnumber — ON DELETE CASCADE ON UPDATE CASCADE

필기 ✍️

✏ 예.풀이 부서가 없어지면 그 부서의 위치 정보도 필요 없음 ⟶ CASCADE

Discussion: Referential Triggered Actions

❑ How to Set Referential Triggered Actions?

■ Consider that the EMPLOYEE table's constraint EMPSUPERFK is changed as follows:

```
CREATE TABLE EMPLOYEE
  ( ... ,
    Dno        INT          NOT NULL      DEFAULT 1,
  CONSTRAINT EMPPK
    PRIMARY KEY (Ssn),
  CONSTRAINT EMPSUPERFK
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
                ON DELETE SET NULL        ON UPDATE CASCADE,
  CONSTRAINT EMPDEPTFK
    FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)
                ON DELETE SET DEFAULT     ON UPDATE CASCADE);
```

The action for **ON DELETE** is changed to CASCADE

```
CONSTRAINT EMPSUPERFK
  FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
              ON DELETE CASCADE ON UPDATE CASCADE,
```

■ Q. What happens when the following command is run?

DELETE EMPLOYEE **WHERE** Lname = 'Borg'

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

13

토론: 참조 트리거 동작

- EMPSUPERFK를 다음처럼 바꿨다고 하자

```
… REFERENCES EMPLOYEE(Ssn) ON DELETE CASCADE
```

- Q. 이 명령을 실행하면?

```
DELETE FROM EMPLOYEE WHERE Lname = 'Borg';
```

필기 ✍️

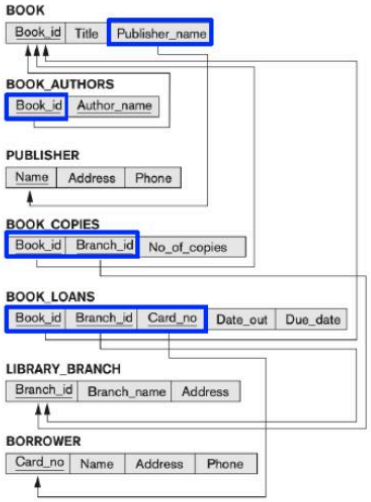
✏ 예.풀이 답: Borg(88866555)를 상사로 둔 Wong·Wallace가 연쇄 삭제 ⟶ 그들을 상사로 둔 직원도 차례로 ⟶ 감독 사슬 전체가 지워짐

⚠ 주의 CASCADE는 연쇄가 이어진다 — 순환 관계에서 특히 위험

Exercises: Specifying Integrity Constraints

❑ **Exercise 1**

- ■ Consider the LIBRARY relational database schema
- ■ Choose the appropriate action (REJECT, CASCADE, SET NULL, SET DEFAULT) for each referential integrity constraint
  - ❑ For the *deletion* of a referenced tuple
  - ❑ For the *update* of a primary key attribute value in a referenced tuple



14

연습 1: 무결성 제약 동작 정하기

- LIBRARY 관계형 스키마에서
- 각 참조 무결성 제약마다 알맞은 동작(REJECT, CASCADE, SET NULL, SET DEFAULT)을 골라라
  - 참조되는 튜플을 **삭제**할 때
  - 참조되는 튜플의 **PK 값을 수정**할 때

필기 ✍️

✏ 예.풀이 정답은 p.64~67 — 대부분 ON UPDATE CASCADE, 삭제는 상황 따라 CASCADE/REJECT

Update Commands in SQL



❑ **Three Commands to Modify the Database**

- ■ **INSERT**: add a single tuple to a relation
- ■ **DELETE**: remove tuples from a relation
- ■ **UPDATE**: modify attribute values of one or more selected tuples

```
INSERT INTO     EMPLOYEE
VALUES          ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
                Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );

DELETE FROM     EMPLOYEE
WHERE           Lname = 'Brown';

UPDATE      PROJECT
SET         Plocation = 'Bellaire', Dnum = 5
WHERE       Pnumber = 10;
```

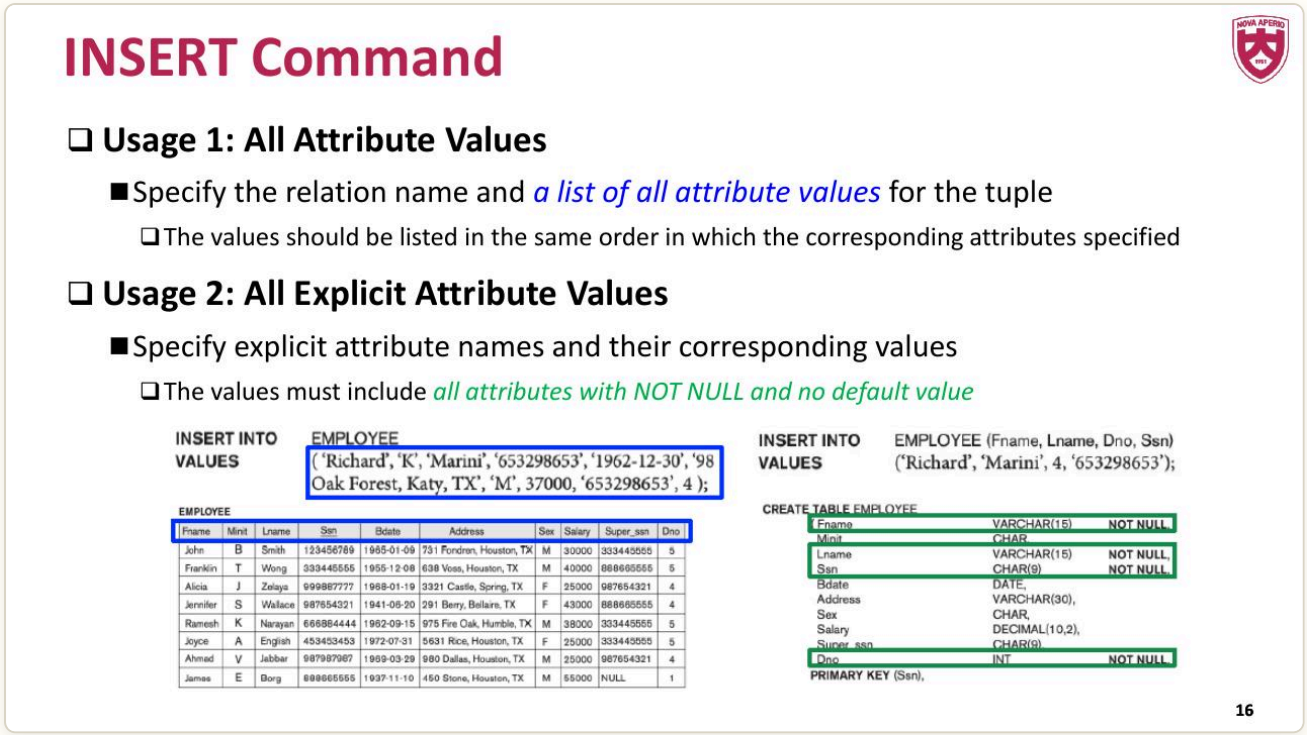
15

SQL 갱신 명령

- **INSERT** — 릴레이션에 튜플 하나 추가
- **DELETE** — 릴레이션에서 튜플 제거
- **UPDATE** — 선택한 튜플(들)의 속성 값 수정

필기 ✍️

— 넘어가도 되는 쪽



INSERT Command

❑ Usage 1: All Attribute Values

■ Specify the relation name and *a list of all attribute values* for the tuple

❑ The values should be listed in the same order in which the corresponding attributes specified

❑ Usage 2: All Explicit Attribute Values

■ Specify explicit attribute names and their corresponding values

❑ The values must include *all attributes with NOT NULL and no default value*

INSERT INTO EMPLOYEE
VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30', '98 Oak Forest, Katy, TX', 'M', 37000, '653298653', 4);

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES ('Richard', 'Marini', 4, '653298653');

CREATE TABLE EMPLOYEE

Fname	VARCHAR(15)	NOT NULL,
Minit	CHAR,	
Lname	VARCHAR(15)	NOT NULL,
Ssn	CHAR(9)	NOT NULL,
Bdate	DATE,	
Address	VARCHAR(30),	
Sex	CHAR,	
Salary	DECIMAL(10,2),	
Super_ssn	CHAR(9),	
Dno	INT	NOT NULL,

PRIMARY KEY (Ssn),

16

INSERT 명령

- **사용법 1: 모든 속성 값** — 정의된 순서대로

```
INSERT INTO EMPLOYEE VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30', '98 Oak
Forest, Katy, TX', 'M', 37000, '653298653', 4);
```

- **사용법 2: 속성 이름을 명시**하고 그 값만
  - **NOT NULL이면서 기본값 없는** 속성은 반드시 넣어야 함

```
INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn) VALUES ('Richard', 'Marini', 4, '653298653');
```

필기 ✍️

★ 시험 사용법 2에서 빠진 속성은 DEFAULT 또는 NULL로 채워짐

INSERT Command (cont.)

❑ Violating Integrity Constraints of INSERT Commands

INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno)
VALUES ('Robert', 'Hatcher', '980760540', 2);

No DEPARTMENT tuple with Dnumber=2 exists

→ Violating the referential integrity constraint
Thus, this should be rejected!

INSERT INTO EMPLOYEE (Fname, Lname, Dno)
VALUES ('Robert', 'Hatcher', 5);

No Ssn value is provided

→ Violating NOT NULL constraint
This should be rejected!

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire

17

INSERT의 무결성 제약 위반

- Dno = 2 삽입 → Dnumber 2 부서가 없음 → **참조 무결성 위반 → 거부**
- Ssn 없이 삽입 ('Robert', 'Hatcher', 5) → **NOT NULL(개체 무결성) 위반 → 거부**

필기 ✍️
— 넘어가도 되는 쪽

DELETE Command

❑ Usage

- ■ **WHERE**: select the tuple(s) to be deleted from **a single relation**
  - ❑ Tuples are explicitly deleted from only one table at a time
  - ❑ The deletion may propagate to tuples in other relations if *referential triggered actions* are specified
    - ■ E.g., **Trigger**: ON DELETE → **Action**: CASCASE
- ■ The number of tuples to be deleted can be different
  - ❑ *Missing **WHERE** deletes all tuples!*

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DELETE FROM **WHERE**	EMPLOYEE Lname = 'Brown';	No tuple is deleted
DELETE FROM **WHERE**	EMPLOYEE Ssn = '123456789';	1 tuple is deleted
DELETE FROM **WHERE**	EMPLOYEE Dno = 5;	4 tuples are deleted
DELETE FROM	EMPLOYEE;	all tuples are deleted

18

DELETE 명령

- **WHERE**로 한 릴레이션에서 지울 튜플 선택
  - 한 번에 **한 테이블**에서만 지운다
  - 참조 트리거 동작(ON DELETE CASCADE)이 있으면 다른 릴레이션으로 **전파**
  - 지워지는 튜플 수는 다양 — Lname='Brown' → 0개 / Ssn='123456789' → 1개 / Dno=5 → 4개
  - **WHERE가 없으면 모든 튜플을 지운다!**

필기 ✍️

⚠ 주의 DELETE FROM EMPLOYEE; ⟶ 표는 남고 행이 전부 사라짐 (DROP TABLE과 다름)

💡 이해 슬라이드 'CASCASE'는 CASCADE 오타

UPDATE Command



❑ Usage

■ **WHERE**: select the tuple(s) to be modified from **a single relation**

❑ The update may propagate to tuples in other relations if *referential triggered actions* are specified

■ E.g., **Trigger**: ON UPDATE → **Action**: CASCASE

■ **SET**: specify the attributes to be modified and their new values

```
UPDATE    PROJECT
SET       Plocation = 'Bellaire', Dnum = 5
WHERE     Pnumber = 10;
```

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

```
UPDATE    EMPLOYEE
SET       Salary = Salary * 1.1
WHERE     Dno = 5;
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Several tuples can be modified with a single UPDATE command

19

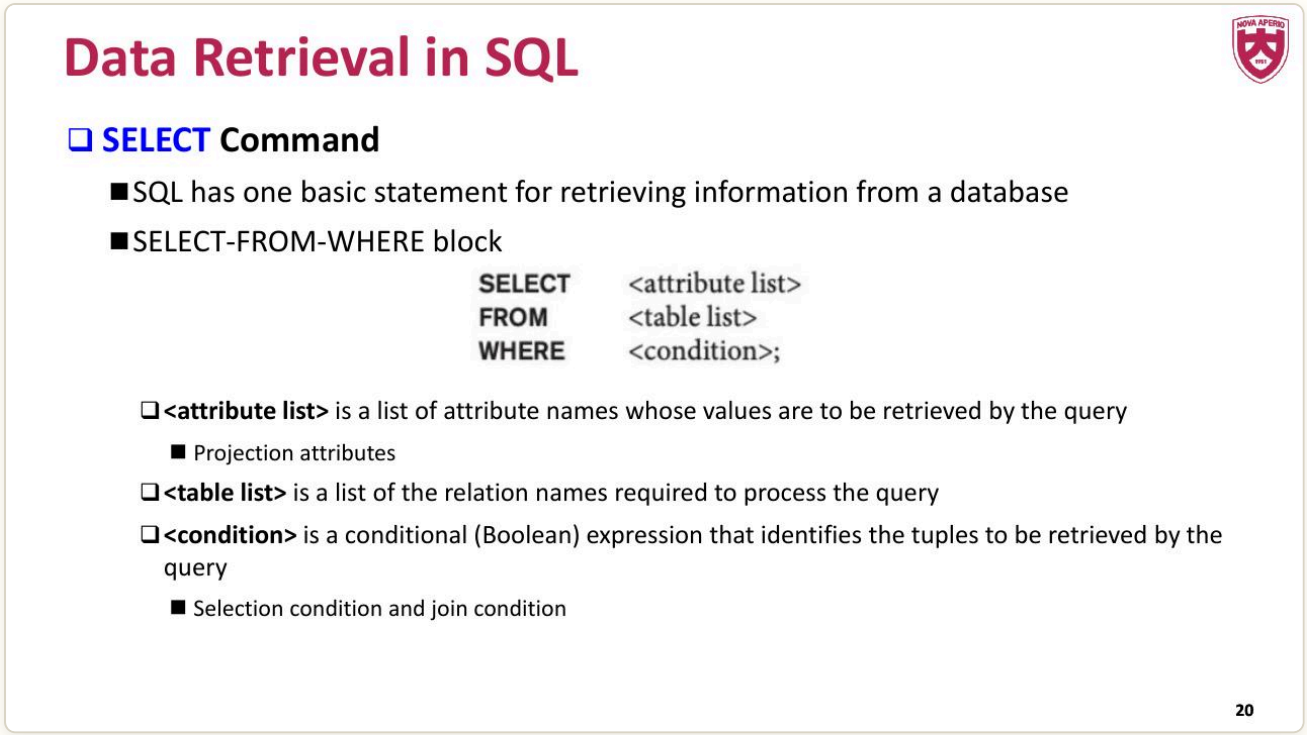
UPDATE 명령

- **WHERE**로 한 릴레이션에서 고칠 튜플 선택
  - ON UPDATE CASCADE면 다른 릴레이션으로 전파
- **SET**으로 바꿀 속성과 새 값 지정

```
UPDATE PROJECT SET Plocation = 'Bellaire', Dnum = 5 WHERE Pnumber = 10;
UPDATE EMPLOYEE SET Salary = Salary * 1.1 WHERE Dno = 5;
```

필기 ✍️

✏ 예.풀이 UPDATE 하나로 여러 튜플 동시 수정 (5번 부서 전원 10% 인상)



Data Retrieval in SQL

❑ SELECT Command

■ SQL has one basic statement for retrieving information from a database

■ SELECT-FROM-WHERE block

```
SELECT   <attribute list>
FROM     <table list>
WHERE    <condition>;
```

❑ **<attribute list>** is a list of attribute names whose values are to be retrieved by the query

■ Projection attributes

❑ **<table list>** is a list of the relation names required to process the query

❑ **<condition>** is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query

■ Selection condition and join condition

20

SQL 데이터 검색 — SELECT

- SQL의 기본 검색문은 하나: **SELECT-FROM-WHERE** 블록

```
SELECT <attribute list> FROM <table list> WHERE <condition>;
```

  - <attribute list>: 가져올 속성 = **projection attributes**
  - <table list>: 질의에 필요한 릴레이션들
  - <condition>: 가져올 튜플을 정하는 Boolean 식 = **selection condition + join condition**

필기 ✍️

★ 시험 SELECT = 관계 대수의 π(project),
WHERE = σ(select) + 조인 조건 — 이름 헷갈림 주의

Data Retrieval in SQL: Examples

❑ **Query 1:**

■ Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'

```
SELECT   Bdate, Address
FROM     EMPLOYEE
WHERE    Fname = 'John' AND Minit = 'B' AND Lname = 'Smith';
```

Projection attributes:
Specified attributes whose values are to be retrieved

Selection condition:
Choose the particular tuples of interest in a table

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Bdate	Address
1965-01-09	731Fondren, Houston, TX

21

검색 예 — Query 1

- 'John B. Smith'의 생일과 주소를 구하라

```
SELECT Bdate, Address FROM EMPLOYEE WHERE Fname = 'John' AND Minit = 'B' AND Lname =
'Smith';
```

- 결과: 1965-01-09, 731 Fondren, Houston, TX

필기 ✍️

— 넘어가도 되는 쪽

Data Retrieval in SQL: Examples (cont.)

❑ **Query 2:**

■ Retrieve the name and address of all employees who work for the 'Research' dept.

```
SELECT   Fname, Lname, Address        Projection attributes
FROM     EMPLOYEE, DEPARTMENT
WHERE    Dname = 'Research' AND Dnumber = Dno;
```

Selection condition

Join condition: Combine two tuples, one from DEPARTMENT and one from EMPLOYEE

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

Fname	Lname	Address
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

22

검색 예 — Query 2

- 'Research' 부서에서 일하는 모든 직원의 이름과 주소

```
SELECT Fname, Lname, Address FROM EMPLOYEE, DEPARTMENT WHERE Dname = 'Research' AND Dnumber = Dno;
```

  - Dname = 'Research' → **selection condition**
  - Dnumber = Dno → **join condition** (DEPARTMENT 튜플과 EMPLOYEE 튜플 결합)
- 결과: John Smith, Franklin Wong, Ramesh Narayan, Joyce English

필기 ✍️

★ 시험 selection 조건과 join 조건을 구분해 쓸 수 있어야 함

Ambiguous Attribute Name

❑ Preventing Ambiguity

- ■ The same name can be used for two (or more) attributes in *different* relations
- ■ We must **qualify** the attribute name with the relation name to prevent ambiguity
  - ❑ If multitable query refers to two or more attributes with the same name

```
SELECT   Fname, EMPLOYEE.Name, Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    DEPARTMENT.Name = 'Research' AND
         DEPARTMENT.Dnumber = EMPLOYEE.Dnumber;
```

Prefixing the relation name to the attribute name and separating the two by a period(.) to prevent ambiguity

```
SELECT   EMPLOYEE.Fname, EMPLOYEE.LName,
         EMPLOYEE.Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    DEPARTMENT.DName = 'Research' AND
         DEPARTMENT.Dnumber = EMPLOYEE.Dno;
```

Fully qualified attribute names can be used for better clarity

EMPLOYEE

Fname	Minit	Name	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dnumber
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Name	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

23

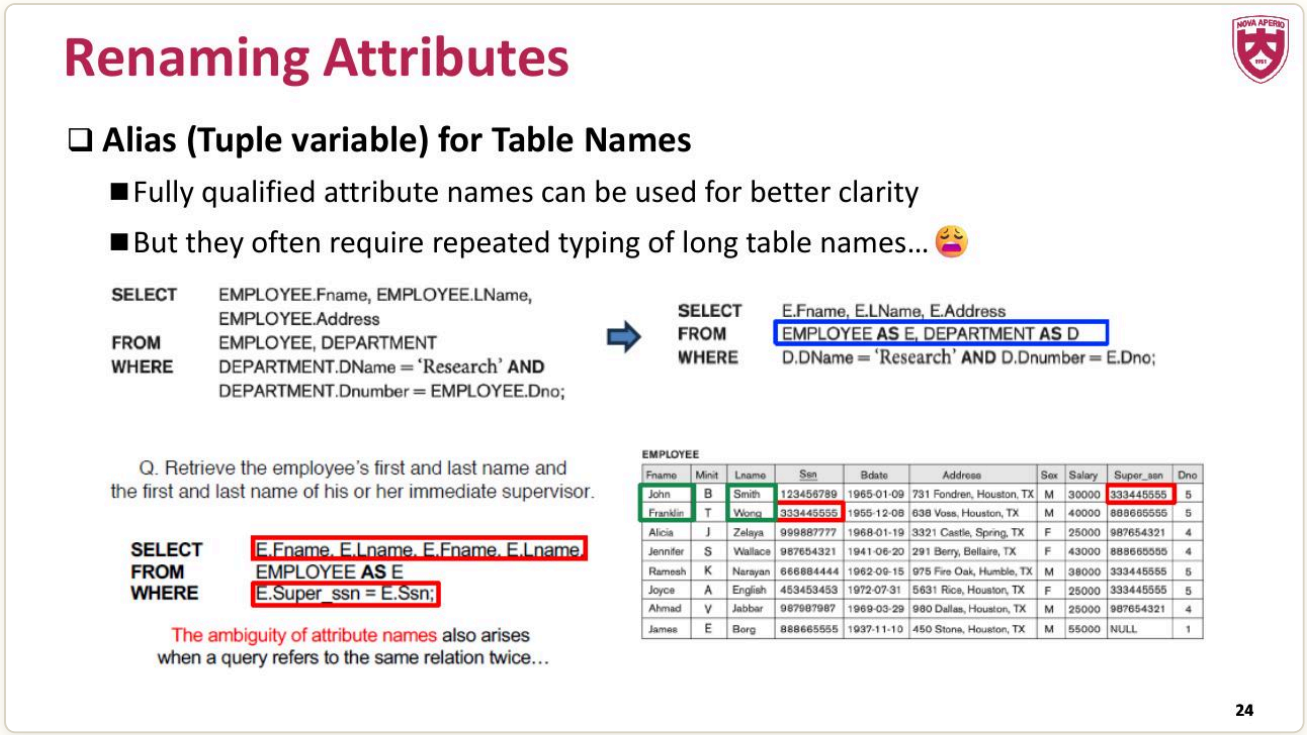
모호한 속성 이름

- 다른 릴레이션의 속성이 **같은 이름**일 수 있다
  - 여러 테이블 질의에서 같은 이름이 둘 이상이면 **릴레이션 이름으로 한정**해야

```
EMPLOYEE.Name, DEPARTMENT.Name  (마침표로 구분)
```

필기 ✍️

— 넘어가도 되는 쪽



Renaming Attributes

❑ Alias (Tuple variable) for Table Names

- ■ Fully qualified attribute names can be used for better clarity
- ■ But they often require repeated typing of long table names… 😩

```
SELECT   EMPLOYEE.Fname, EMPLOYEE.LName,
         EMPLOYEE.Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    DEPARTMENT.DName = 'Research' AND
         DEPARTMENT.Dnumber = EMPLOYEE.Dno;
```

⇒

```
SELECT   E.Fname, E.LName, E.Address
FROM     EMPLOYEE AS E, DEPARTMENT AS D
WHERE    D.DName = 'Research' AND D.Dnumber = E.Dno;
```

Q. Retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT   E.Fname, E.Lname, E.Fname, E.Lname
FROM     EMPLOYEE AS E
WHERE    E.Super_ssn = E.Ssn;
```

The ambiguity of attribute names also arises when a query refers to the same relation twice…

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

24

별칭 (Alias, tuple variable)

- 완전히 한정된 이름은 명확하지만 긴 테이블 이름을 반복해야 함
  - → 테이블 이름에 **별칭** — FROM EMPLOYEE AS E, DEPARTMENT AS D
- 같은 표를 **두 번** 쓸 때도 필요: 직원과 직속 상사의 이름

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname FROM EMPLOYEE AS E, EMPLOYEE AS S WHERE
E.Super_ssn = S.Ssn;
```

필기 ✍️

★ 시험 자기 자신과 조인(순환 관계)은 별칭 2개 필수

Two More Features of SQL

❑ Missing WHERE Clause

- ■ No condition on tuple selection
- ■ Retrieve ***all tuples*** of the relations specified in FROM

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

```
SELECT  Ssn
FROM    EMPLOYEE;
```

Ssn
123456789
333445555
999887777
987654321
666884444
453453453
987987987
888665555

```
SELECT  Ssn, Dname
FROM    EMPLOYEE, DEPARTMENT;
```

Ssn	Dname
123456789	Research
333445555	Research
999887777	Research
987654321	Research
666884444	Research
453453453	Research
987987987	Research
888665555	Research
123456789	Administration
333445555	Administration
999887777	Administration
987654321	Administration
666884444	Administration
453453453	Administration
987987987	Administration
888665555	Administration
123456789	Headquarters
333445555	Headquarters
999887777	Headquarters
987654321	Headquarters
666884444	Headquarters
453453453	Headquarters
987987987	Headquarters
888665555	Headquarters

It is extremely important to specify every selection and join condition in the WHERE Clause!
if any such condition is overlooked, incorrect and very large relations may result

All possible tuple combinations
3*8=24

25

SQL의 두 가지 특징 ① WHERE 생략

- 튜플 선택 조건이 없음 → FROM 릴레이션의 **모든 튜플**

```
SELECT Ssn FROM EMPLOYEE;  → 8행
```

- 두 릴레이션에서 WHERE를 빼면 **모든 조합**(카티션 곱)

```
SELECT Ssn, Dname FROM EMPLOYEE, DEPARTMENT;  → 8 × 3 = 24행
```

- → selection·join 조건을 WHERE에 **빠짐없이** 쓰는 게 매우 중요

필기 ✍️

⚠ 주의 조인 조건 빼먹음 = 틀리고 거대한 결과 (서술형 감점 1순위)

Two More Features of SQL (cont.)



❑ Use of the Asterisk (*)

- ■ Retrieve ***all the attributes values*** of the selected tuples
- ■ To avoid inconvenience of specifying all attribute names
  - ❑ Do not have to list the attribute names explicitly in SQL

```
SELECT   *
FROM     EMPLOYEE
WHERE    Dno = 5;
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-09-01	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

26

SQL의 두 가지 특징 ② 별표(*)

- 선택된 튜플의 **모든 속성** 값
  - 속성 이름을 다 적는 불편을 피함

```
SELECT * FROM EMPLOYEE WHERE Dno = 5;
```

필기 ✍️

— 넘어가도 되는 쪽

Tables as (Multi)Sets



❑ Tables as Multisets in SQL

- ■ SQL treats a table not as a *set* but rather as a ***multiset***
  - ❑ i.e., "duplicate tuples" can exist in a table (or in the result of a query)
- ■ SQL *does not automatically eliminate duplicate tuples* in query results

❑ Why SQL treats Tables as Multisets?

- ■ Duplicate elimination is an expensive operation
  - ❑ E.g., sorting the tuples first and eliminate duplicates whenever new tuples are inserted
- ■ The user may want to see duplicate tuples in the result of a query
- ■ When an *aggregate function* is applied to tuples, duplicates may be needed

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

SELECT ALL Salary
FROM EMPLOYEE;

Salary
30000
40000
25000
43000
38000
25000
25000
55000

27

다중집합으로서의 테이블

- SQL은 테이블을 집합이 아니라 **multiset**으로 다룬다
  - 테이블(또는 결과)에 **중복 튜플**이 있을 수 있음
  - 결과에서 중복을 **자동으로 없애지 않는다**
- 왜?
  - 중복 제거는 **비싸다** (정렬 후 제거)
  - 사용자가 중복을 **보고 싶을** 수 있다
  - **집계 함수**에는 중복이 필요할 수 있다

필기 ✍️

★ 시험 이유 3개: 비용 · 사용자 의도 · 집계계

⚠ 주의 관계형 모델의 릴레이션은 set(중복 없음), SQL 테이블은 multiset — 대비 문제

Tables as (Multi)Sets (Cont.)



❑ **ALL Keyword**

■ Does not eliminate duplicate tuples from the result of a query

❑ **DISTINCT Keyword**

■ Eliminate duplicate tuples from the result of an SQL query

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Q1. Retrieve the salary of every employee
Q2. Retrieve all distinct salary values

Q1 **SELECT** **ALL** Salary
FROM EMPLOYEE;

Q2 **SELECT** **DISTINCT** Salary
FROM EMPLOYEE;

Salary
30000
40000
25000
43000
38000
25000
25000
55000

Q1

Salary
30000
40000
25000
43000
38000
55000

Q2

28

ALL과 DISTINCT

- **ALL**: 중복 제거 안 함 (기본)
- **DISTINCT**: 결과에서 중복 제거

```
SELECT ALL Salary FROM EMPLOYEE;  → 30000, 40000, 25000, 43000, 38000, 25000, 25000, 55000
SELECT DISTINCT Salary FROM EMPLOYEE;  → 30000, 40000, 25000, 43000, 38000, 55000
```

필기 ✍️

— 넘어가도 되는 쪽

Tables as (Multi)Sets (Cont.)

❑ Set Operations

- ■ Set union, (**UNION**), set difference (**EXCEPT**), set intersection (**INTERSECT**)
- ■ The relations resulting from these set operations are ***sets*** of tuples
  - ❑ That is, duplicate tuples are eliminated from the result
- ■ They apply only to ***type-compatible*** relations

Query 4. Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
( SELECT    DISTINCT Pnumber
  FROM      PROJECT, DEPARTMENT, EMPLOYEE
  WHERE     Dnum = Dnumber AND Mgr_ssn = Ssn
            AND Lname = 'Smith' )
  UNION
( SELECT    DISTINCT Pnumber
  FROM      PROJECT, WORKS_ON, EMPLOYEE
  WHERE     Pnumber = Pno AND Essn = Ssn
            AND Lname = 'Smith' );
```

Projects in which 'Smith' works as a manager

Projects in which 'Smith' works as a worker

29

집합 연산

- **UNION**(합), **EXCEPT**(차), **INTERSECT**(교)
  - 결과는 **튜플의 집합** — 중복 제거됨
  - **type-compatible** 릴레이션에만 적용
- 예(Query 4): 'Smith'가 일꾼이거나 관리 부서의 관리자인 프로젝트 번호

```
(SELECT DISTINCT Pnumber FROM PROJECT, DEPARTMENT, EMPLOYEE WHERE Dnum = Dnumber AND
Mgr_ssn = Ssn AND Lname = 'Smith') UNION (SELECT DISTINCT Pnumber FROM PROJECT, WORKS_ON,
EMPLOYEE WHERE Pnumber = Pno AND Essn = Ssn AND Lname = 'Smith');
```

필기 ✍️

★ 시험 집합 연산은 기본이 중복 제거! (SELECT와 반대)

Tables as (Multi)Sets (Cont.)

❑ **Multiset Operations**

■ **UNION ALL, EXCETP ALL, INTERSECT ALL**

■ SQL also has corresponding multiset operations, followed by the keyword **ALL**

(a) R

A
a1
a2
a2
a3

S

A
a1
a2
a4
a5

(b) T

A
a1
a1
a2
a2
a2
a3
a4
a5

R(A) **UNION ALL** R(S)

(c) T

A
a2
a3

R(A) **EXCEPT ALL** R(S)

(d) T

A
a1
a2

R(A) **INTERSECT ALL** R(S)

30

다중집합 연산

- **UNION ALL, EXCEPT ALL, INTERSECT ALL** — 중복 유지
- 예: R(A) = {a1, a2, a2, a3}, S(A) = {a1, a2, a4, a5}
  - UNION ALL → a1, a1, a2, a2, a2, a3, a4, a5
  - EXCEPT ALL → a2, a3
  - INTERSECT ALL → a1, a2

필기 ✍️

✏ 예.풀이 EXCEPT ALL: a2가 R에 2개, S에 1개
⟶ 하나 남음

💡 이해 슬라이드 'EXCETP'는 EXCEPT 오타

Ordering of Query Results in SQL

❑ ORDER BY Clause

- ■ Order tuples in the result of a query by the values of one or more of the attributes
- ■ The default order is in ascending order of values
  - ❑ **DESC** (Descending order) and **ASC** (ascending order) keywords

Query 15. Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, then first name.

```
Q15:  SELECT    D.Dname, E.Lname, E.Fname, P.Pname
      FROM      DEPARTMENT AS D, EMPLOYEE AS E, WORKS_ON AS W,
                PROJECT AS P
      WHERE     D.Dnumber = E.Dno AND E.Ssn = W.Essn AND W.Pno =
                P.Pnumber
      ORDER BY  D.Dname, E.Lname, E.Fname;
```

Or

```
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC
```

31

결과 정렬 — ORDER BY

- 하나 이상 속성 값으로 결과를 정렬
  - 기본은 **오름차순** — DESC(내림) / ASC(오름)

```
… ORDER BY D.Dname, E.Lname, E.Fname;
… ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC;
```

필기 ✍️

— 넘어가도 되는 쪽

Standard Arithmetic Operators

❑ **Arithmetic Operators Can be Applied to Numeric Attributes**

■ Addition (+), Subtraction (-), multiplication (*), division (/)

Query 13. Show the resulting salaries if every employee working on the 'ProductX' project is given a 10% raise.

```
Q13:  SELECT   E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
      FROM     EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
      WHERE    E.Ssn = W.Essn AND W.Pno = P.Pnumber AND
               P.Pname = 'ProductX';
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

30,000*1.1 = 33,000
25,000*1.1 = 27,500

32

산술 연산자

- 숫자 속성에 +, −, *, / 사용 가능
- 예(Query 13): ProductX 직원에게 10% 인상하면 월급은?

```
SELECT E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal FROM EMPLOYEE AS E, WORKS_ON AS W,
PROJECT AS P WHERE E.Ssn = W.Essn AND W.Pno = P.Pnumber AND P.Pname = 'ProductX';
```

필기 ✍️

✏ 예.풀이 AS로 결과 열 이름도 바꿀 수 있다 (Increased_sal)

Comparison Operators

❑ BETWEEN Operator

■ Specify the range of attribute values in the WHERE clause

Query 14. Retrieve all employees in department 5 whose salary is between $30,000 and $40,000.

```
Q14:  SELECT  *
      FROM    EMPLOYEE
      WHERE   (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

```
((Salary >= 30000) AND (Salary <= 40000))
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

33

비교 연산자 — BETWEEN

- WHERE에서 값 범위 지정

```
SELECT * FROM EMPLOYEE WHERE (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

  - = (Salary >= 30000) AND (Salary <= 40000)

필기 ✍️

⚠ 주의 BETWEEN은 양 끝 포함

Exercises: Data Retrieval in SQL

❑ Exercise 2: Specify the Following Queries in SQL

■ Q. Retrieve the names of all employees in department 5 who work more than 10 hours per week on the "ProductX" project

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

34

연습 2: SQL로 쓰기

- 5번 부서에서 'ProductX' 프로젝트에 주당 **10시간 넘게** 일하는 직원들의 이름

필기 ✍️

✏ 예.풀이 정답 p.68 — 조건 5개(선택 3 + 조인 2)

Exercises: Data Retrieval in SQL

❑ Exercise 3: Specify the Following Queries in SQL

■ Q. Retrieve the name and transcript of each senior student (Class=4) majoring in CS. A transcript includes 1) course name, 2) course number, 3) credit hours, 4) semester, 5) year, and 6) grade for each course completed by the student

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

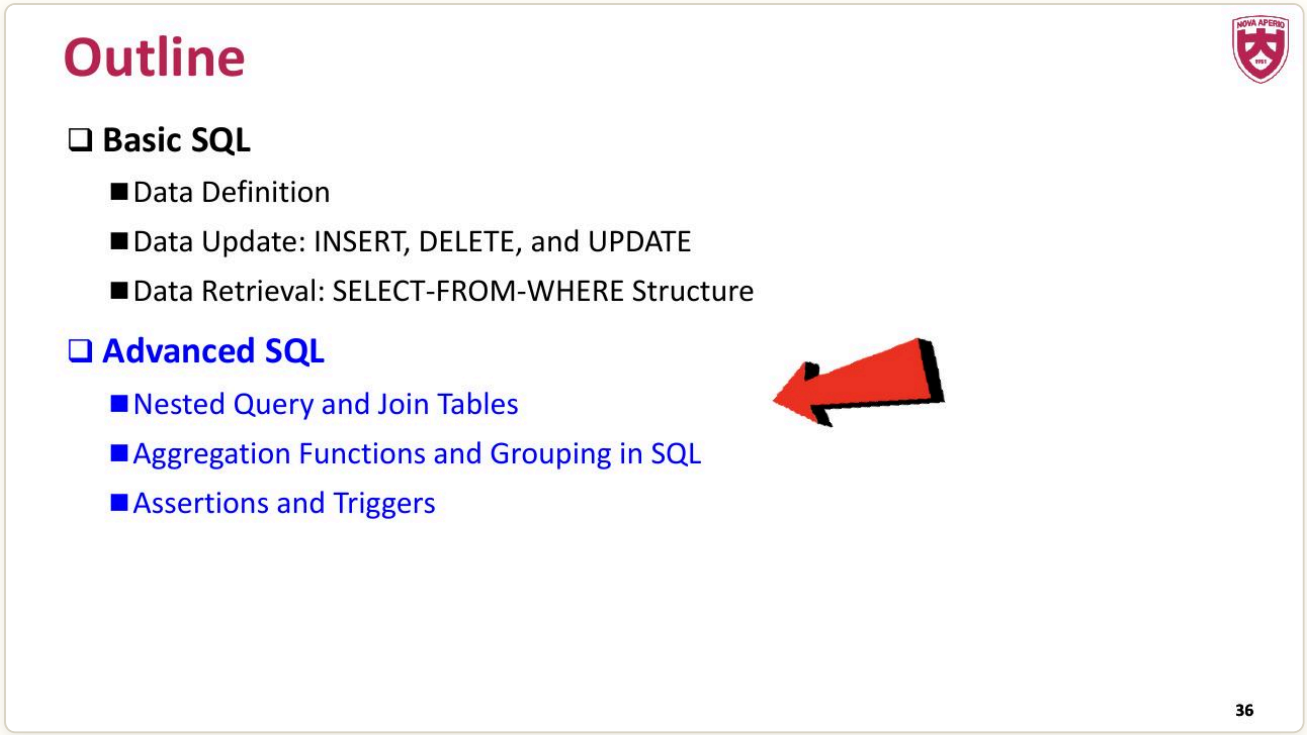
35

연습 3: SQL로 쓰기

- CS 전공 **4학년(Class=4)** 학생의 이름과 성적표
  - 성적표 = 과목명, 과목 번호, 학점, 학기, 연도, 성적 (이수한 과목마다)

필기 ✍️

✏ 예.풀이 정답 p.69 — 표 4개 조인(STUDENT, GRADE_REPORT, SECTION, COURSE)



목차 (Outline)

- **Basic SQL** — 데이터 정의 / 갱신: INSERT·DELETE·UPDATE / 검색: SELECT-FROM-WHERE
- **Advanced SQL** — 중첩 질의와 조인 테이블 / 집계 함수와 그룹화 / 어서션과 트리거

필기 ✍️

💡 이해 후반부: 고급 SQL

Nested Query

❑ What is a Nested Query?

■ Complete SELECT-FROM-WHERE Blocks within a WHERE clause of another query

❑ Useful to fetch existing values in a database and then use them for a comparison in a WHERE condition

■ Returns a **(multi)set of tuples**

Query 4. Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

Outer query

```
SELECT   DISTINCT Pnumber
FROM     PROJECT
WHERE    Pnumber IN                                   Inner query
         ( SELECT   Pnumber
           FROM     PROJECT, DEPARTMENT, EMPLOYEE
           WHERE    Dnum = Dnumber AND
                    Mgr_ssn = Ssn AND Lname = 'Smith' )
         OR
         Pnumber IN                                   Inner query
         ( SELECT   Pno
           FROM     WORKS_ON, EMPLOYEE
           WHERE    Essn = Ssn AND Lname = 'Smith' );
```

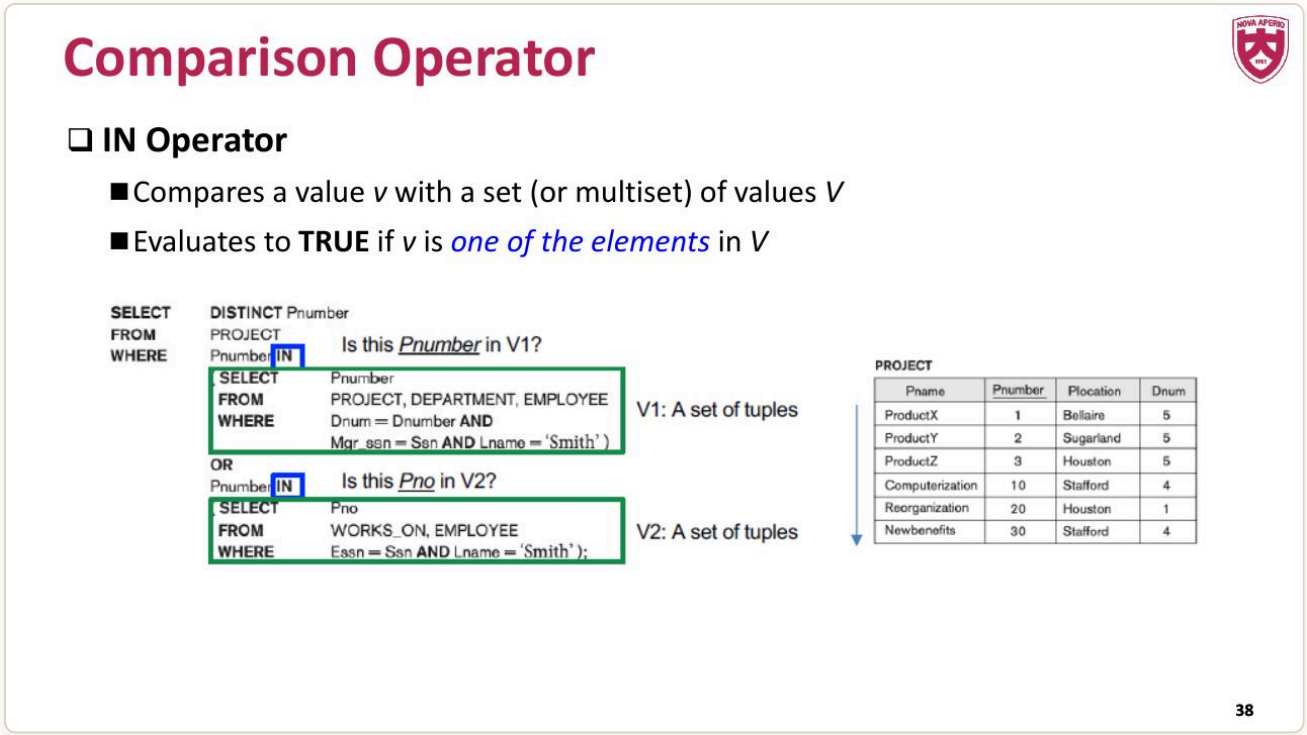
37

중첩 질의 (Nested Query)

- 다른 질의의 **WHERE 절 안에 있는 완전한 SELECT-FROM-WHERE 블록**
  - DB의 기존 값을 가져와 WHERE 비교에 쓸 때 유용
  - (다중)집합의 튜플을 돌려줌
- 예: Query 4를 중첩 질의로 — Pnumber IN (...) OR Pnumber IN (...)

필기 ✍️

— 넘어가도 되는 쪽



비교 연산자 — IN

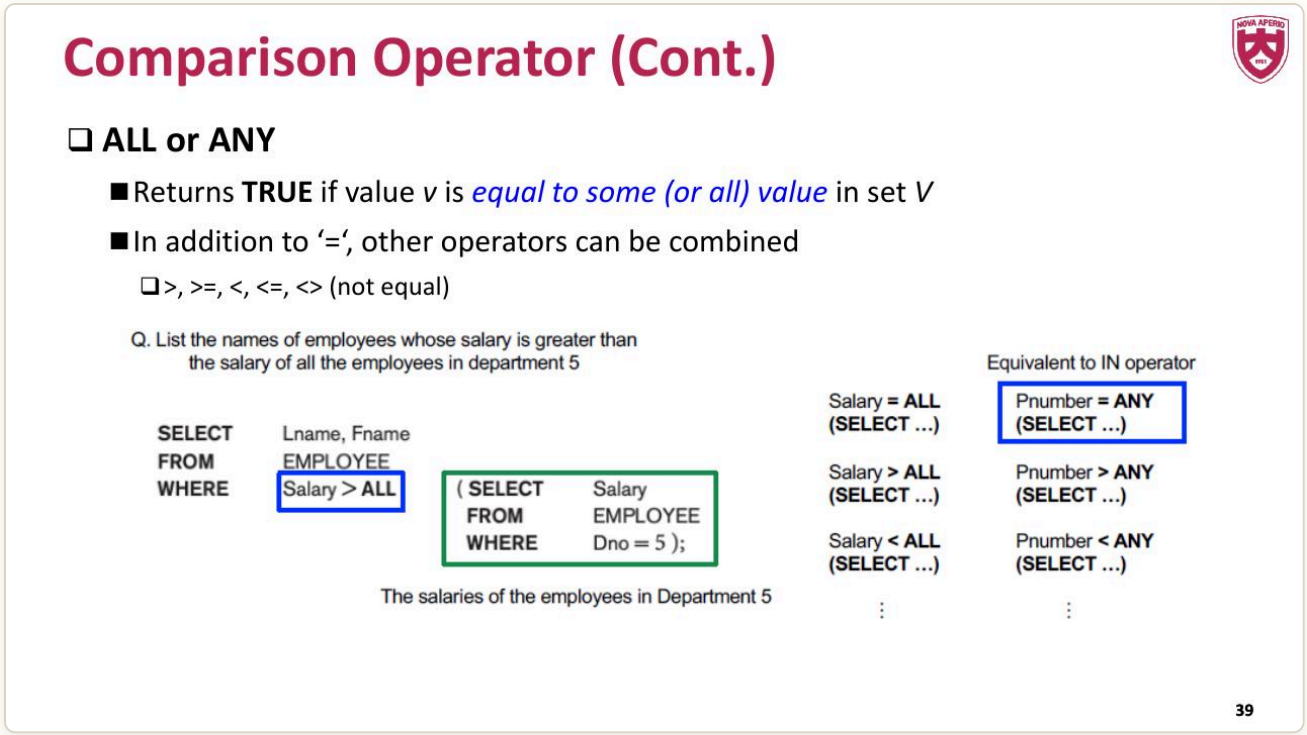
- 값 v를 값의 (다중)집합 V와 비교
  - v가 V의 원소 중 하나면 **TRUE**

```
SELECT DISTINCT Pnumber FROM PROJECT WHERE Pnumber IN (SELECT Pnumber FROM PROJECT,
DEPARTMENT, EMPLOYEE WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith') OR Pnumber
IN (SELECT Pno FROM WORKS_ON, EMPLOYEE WHERE Essn = Ssn AND Lname = 'Smith');
```

  - V1, V2 = 안쪽 질의가 돌려준 튜플 집합 — 이 Pnumber가 V1(또는 V2)에 있나?

필기 ✍️

💡 이해 PROJECT 행을 위에서부터 하나씩 보며 'V1에 있나?'를 묻는 그림



비교 연산자 — ALL, ANY

- v가 V의 **일부(ANY)** 또는 **모든(ALL)** 값과 같으면 TRUE
  - '=' 말고도 >, >=, <, <=, <> 결합 가능

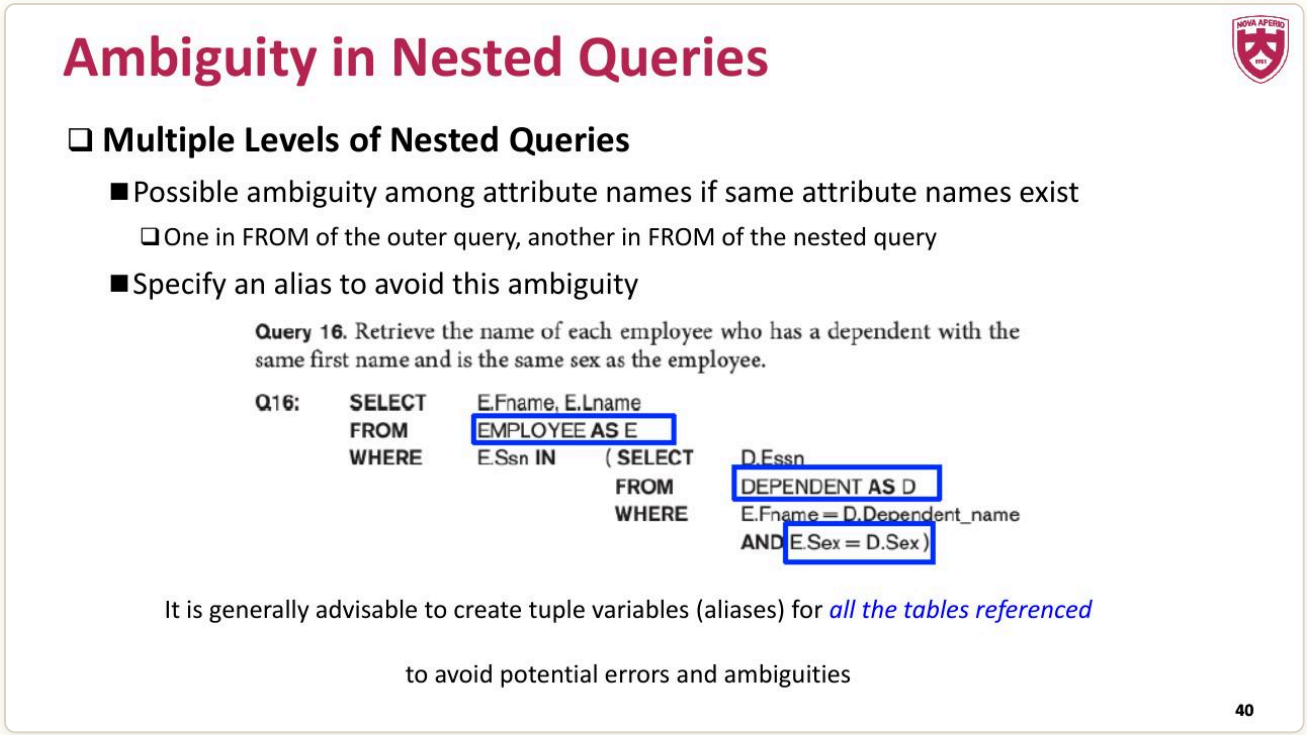
```
SELECT Lname, Fname FROM EMPLOYEE WHERE Salary > ALL (SELECT Salary FROM EMPLOYEE WHERE Dno = 5);
```

  - → 5번 부서 **모든** 직원보다 월급이 많은 직원

필기 ✍️

★ 시험 = ANY ≡ IN

✏ 예.풀이 > ALL = 최댓값보다 큼 / > ANY = 최솟값보다 큼



Ambiguity in Nested Queries

❑ **Multiple Levels of Nested Queries**

■ Possible ambiguity among attribute names if same attribute names exist

❑ One in FROM of the outer query, another in FROM of the nested query

■ Specify an alias to avoid this ambiguity

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
Q16:  SELECT   E.Fname, E.Lname
      FROM     EMPLOYEE AS E
      WHERE    E.Ssn IN   ( SELECT   D.Essn
                            FROM     DEPENDENT AS D
                            WHERE    E.Fname = D.Dependent_name
                            AND E.Sex = D.Sex )
```

It is generally advisable to create tuple variables (aliases) for *all the tables referenced*

to avoid potential errors and ambiguities

40

중첩 질의의 모호성

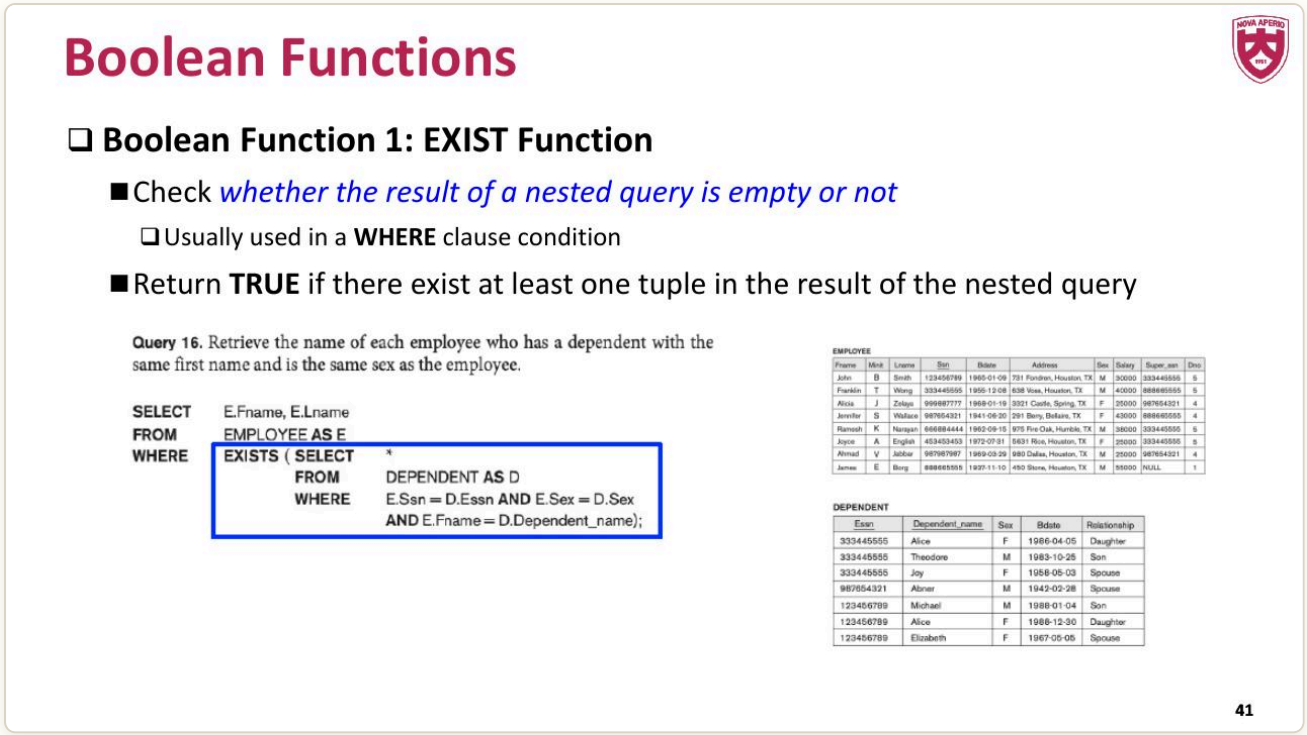
- 여러 단계 중첩에서 같은 속성 이름이 있으면 모호 — 바깥 FROM의 것 vs 안쪽 FROM의 것
  - **별칭**으로 해결

```
SELECT E.Fname, E.Lname FROM EMPLOYEE AS E WHERE E.Ssn IN (SELECT D.Essn FROM DEPENDENT AS
D WHERE E.Fname = D.Dependent_name AND E.Sex = D.Sex);
```

- 참조하는 **모든 테이블에 별칭**을 만드는 것이 일반적으로 좋다

필기 ✍️

💡 이해 안쪽 질의가 바깥 E를 쓰면 '상관(correlated) 중첩 질의' — 바깥 행마다 다시 계산



Boolean Functions

❑ Boolean Function 1: EXIST Function

- Check *whether the result of a nested query is empty or not*
  - ❑ Usually used in a **WHERE** clause condition
- Return **TRUE** if there exist at least one tuple in the result of the nested query

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
SELECT   E.Fname, E.Lname
FROM     EMPLOYEE AS E
WHERE    EXISTS ( SELECT   *
                  FROM     DEPENDENT AS D
                  WHERE    E.Ssn = D.Essn AND E.Sex = D.Sex
                           AND E.Fname = D.Dependent_name);
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

41

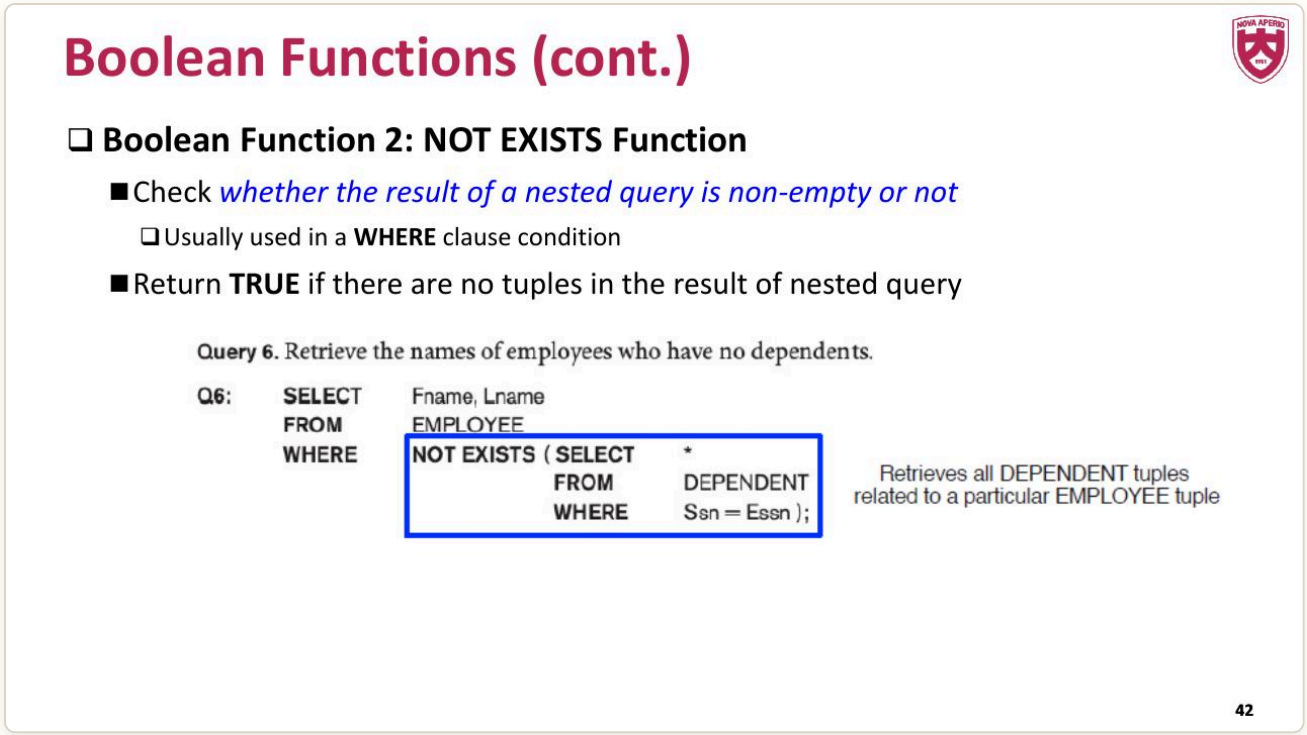
Boolean 함수 ① EXISTS

- 중첩 질의 결과가 비었는지 확인 — 보통 WHERE 조건에
  - 결과에 튜플이 **하나라도 있으면 TRUE**

```
… WHERE EXISTS (SELECT * FROM DEPENDENT AS D WHERE E.Ssn = D.Essn AND E.Sex = D.Sex AND
E.Fname = D.Dependent_name);
```

필기 ✍️

— 넘어가도 되는 쪽



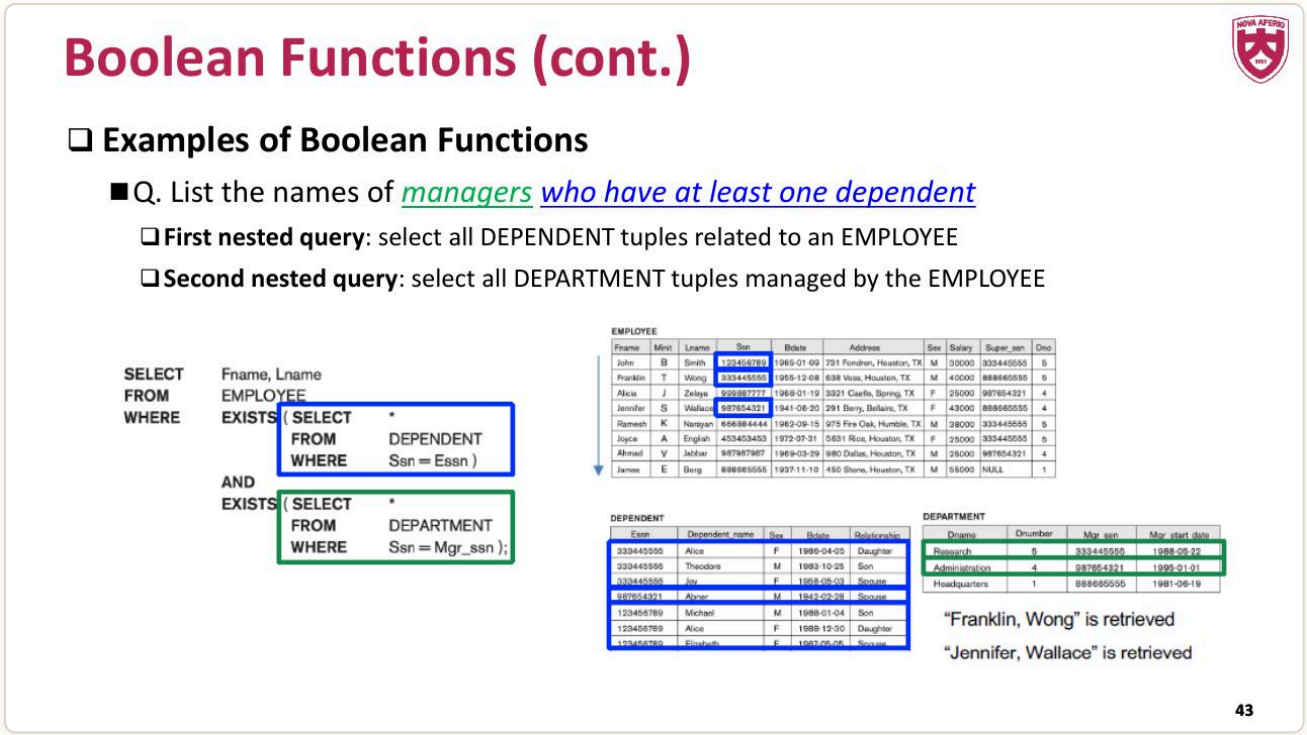
Boolean 함수 ② NOT EXISTS

- 중첩 질의 결과에 튜플이 **없으면 TRUE**
- 예(Query 6): 부양가족이 없는 직원

```
SELECT Fname, Lname FROM EMPLOYEE WHERE NOT EXISTS (SELECT * FROM DEPENDENT WHERE Ssn = Essn);
```

필기 ✍️

💡 이해 슬라이드 설명의 'non-empty'는 empty의 뜻 — 결과가 비면 TRUE



Boolean 함수 예

- Q. 부양가족이 **한 명 이상** 있는 **관리자**의 이름

```
SELECT Fname, Lname FROM EMPLOYEE WHERE EXISTS (SELECT * FROM DEPENDENT WHERE Ssn = Essn)
AND EXISTS (SELECT * FROM DEPARTMENT WHERE Ssn = Mgr_ssn);
```

  - 첫째 중첩: 그 직원의 DEPENDENT 튜플들
  - 둘째 중첩: 그 직원이 관리하는 DEPARTMENT 튜플들

필기 ✍️

✏ 예.풀이 결과: Franklin Wong, Jennifer Wallace

Exercise: Boolean Functions

❑ Exercise 4: Specify the Following Query

■ Q. Retrieve the name of each employee who works on all the projects controlled by department number 5

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

44

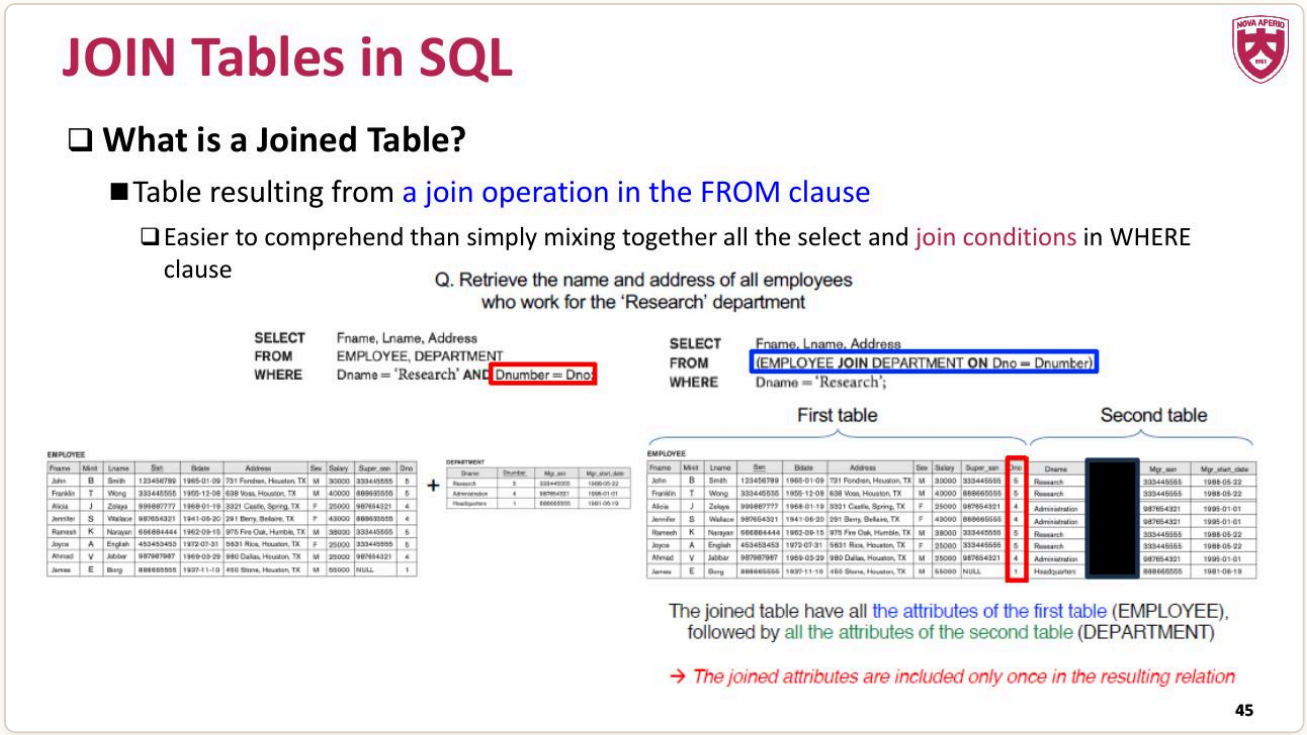
연습 4: Boolean 함수

- 5번 부서가 통제하는 **모든** 프로젝트에서 일하는 직원의 이름

필기 ✍️

✏ 예.풀이 정답 p.70 — NOT EXISTS (5번 부서 프로젝트 EXCEPT 내가 일하는 프로젝트)

★ 시험 '모든 ~에' = 나눗셈 유형 ⟶ NOT EXISTS + EXCEPT (6강 DIVISION과 같은 뜻)



조인 테이블 (Joined Table)

- FROM 절의 **조인 연산**으로 만든 테이블
  - WHERE에 선택·조인 조건을 섞는 것보다 **이해하기 쉽다**

```
SELECT Fname, Lname, Address FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber) WHERE Dname = 'Research';
```

  - 결과: EMPLOYEE의 모든 속성 다음에 DEPARTMENT의 모든 속성

필기 ✍️

— 넘어가도 되는 쪽

Inner Join

❑ What is Inner Join?

■ Default type of join is called an **inner join**,

❑ A tuple is included in the result only if a matching tuple exists in the other relation

```
SELECT   E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM     EMPLOYEE AS E, EMPLOYEE AS S
WHERE    E.Super_ssn = S.Ssn;
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

In the inner join,
only pairs of tuples that match the join condition are retrieved

Ssn	Super_ssn
123456789	333445555
333445555	888665555
999887777	987654321
987654321	888665555
666884444	333445555
453453453	333445555
987987987	987654321

Only employees who have a supervisor are retrieved.
A tuple whose value for Super_ssn is NULL is excluded.

Q. What if we want that all employees are retrieved?

→ OUTER JOIN!

46

내부 조인 (Inner Join)

- 기본 조인 = **inner join**
  - 다른 릴레이션에 **짝이 있을 때만** 결과에 포함

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name FROM EMPLOYEE AS E, EMPLOYEE AS S WHERE E.Super_ssn = S.Ssn;
```

  - Super_ssn이 NULL인 Borg는 결과에서 빠짐
  - Q. 모든 직원을 가져오려면? → OUTER JOIN

필기 ✍️

— 넘어가도 되는 쪽

Outer Join

❑ What is Outer Join?

■ A different type of join, used to retrieve tuples if it does not have a matching tuple

❑ It is padded with NULL values for the attributes

```
SELECT   E.Lname AS Employee_name,
         S.Lname AS Supervisor_name
FROM     (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
         ON E.Super_ssn = S.Ssn);
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Ssn	Super_ssn
123456789	333445555
333445555	888665555
999887777	987654321
987654321	888665555
666884444	333445555
453453453	333445555
987987987	987654321
888665555	NULL

In LEFT OUTER JOIN,
every tuple in the left table must appear in the result.

47

외부 조인 (Outer Join)

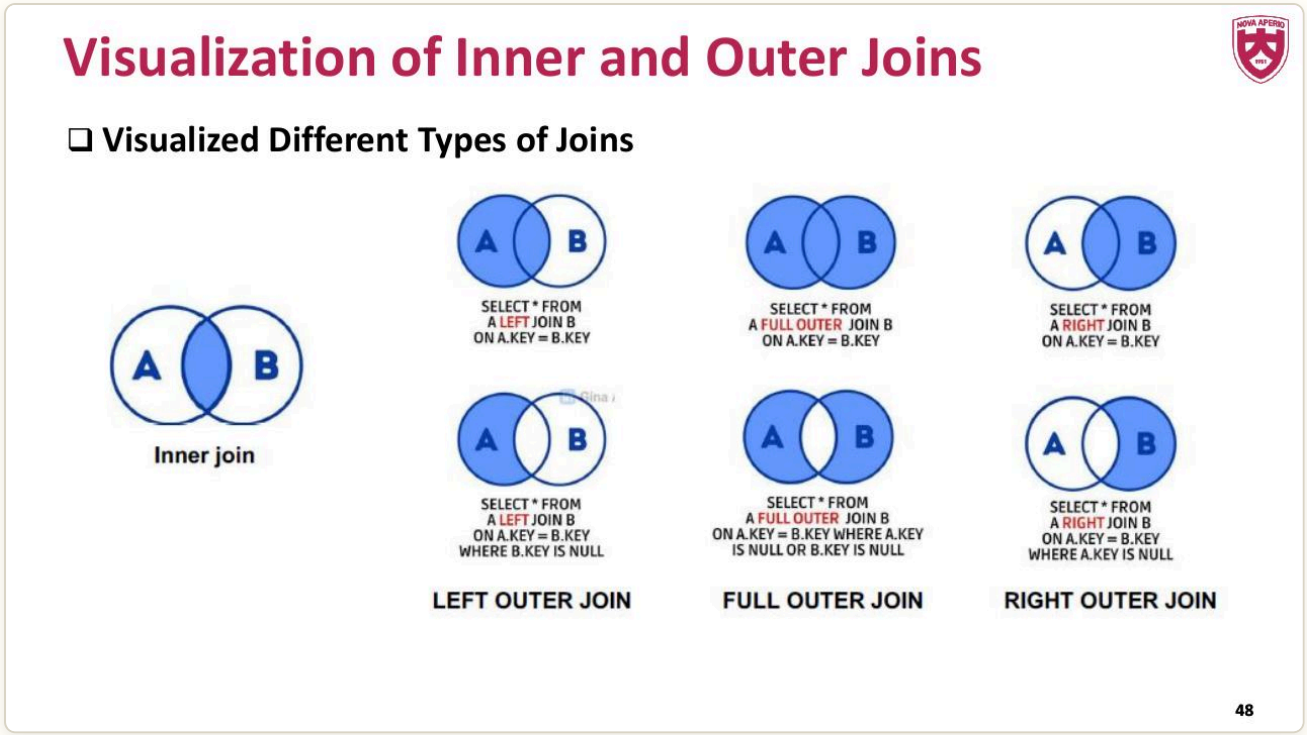
- 짝이 **없어도** 튜플을 가져오는 조인
  - 없는 쪽 속성은 **NULL로 채움**

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name FROM (EMPLOYEE AS E LEFT OUTER
JOIN EMPLOYEE AS S ON E.Super_ssn = S.Ssn);
```

  - LEFT OUTER JOIN: **왼쪽 테이블의 모든 튜플**이 결과에 — Borg는 상사 칸 NULL

필기 ✍️

★ 시험 LEFT = 왼쪽 전부 / RIGHT = 오른쪽 전부 / FULL = 양쪽 전부

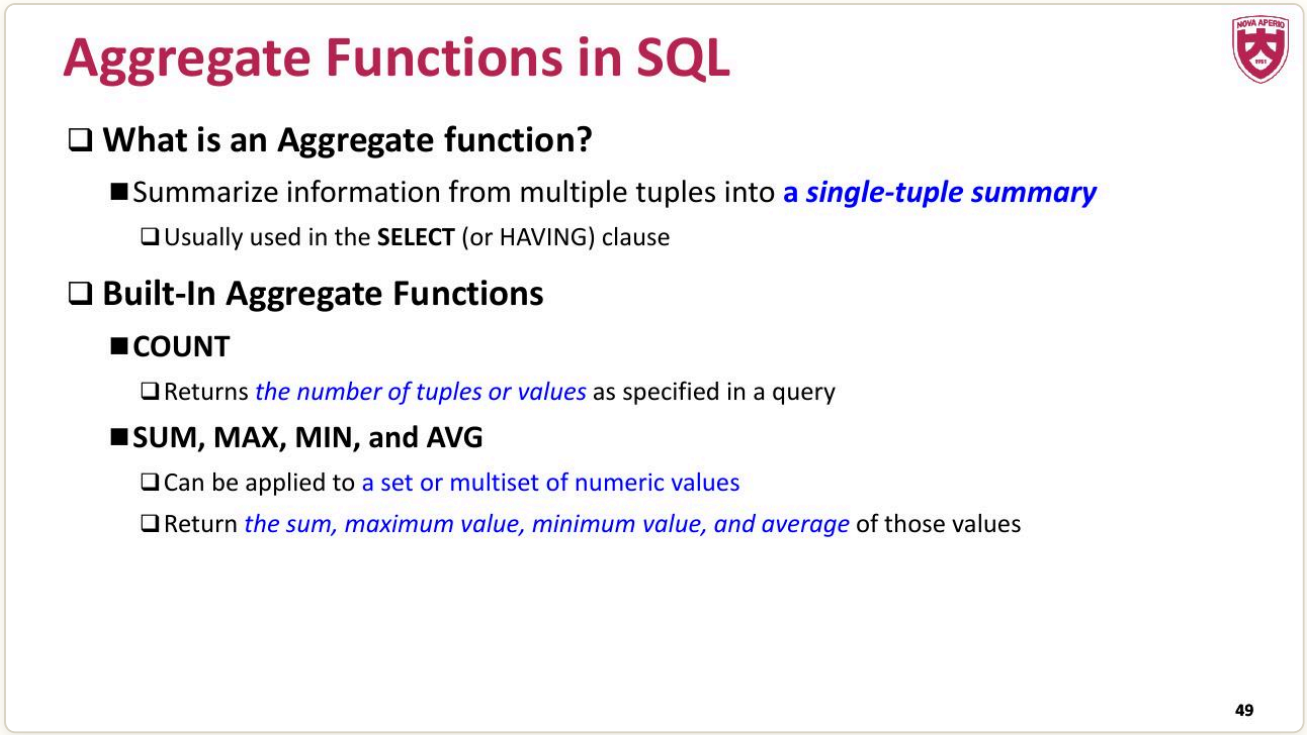


내부·외부 조인 그림

- 벤 다이어그램으로 본 조인 종류
  - INNER JOIN(교집합) / LEFT JOIN / RIGHT JOIN / FULL OUTER JOIN
  - LEFT JOIN ... WHERE B.KEY IS NULL → A에만 있는 것
  - RIGHT JOIN ... WHERE A.KEY IS NULL → B에만 있는 것
  - FULL OUTER JOIN ... WHERE A.KEY IS NULL OR B.KEY IS NULL → 한쪽에만 있는 것

필기 ✍️

💡 이해 그림 개념을 SQL과 짝지어 외우기



집계 함수 (Aggregate Functions)

- 여러 튜플의 정보를 **한 튜플 요약**으로
  - 보통 SELECT(또는 HAVING) 절에
- 기본 집계 함수
  - **COUNT** — 튜플이나 값의 개수
  - **SUM, MAX, MIN, AVG** — 숫자 값의 (다중)집합에 — 합·최대·최소·평균

필기 ✍️

⚠ 주의 집계는 WHERE에 못 쓴다 → HAVING 또는 중첩 질의

Aggregate Functions in SQL (cont.)



❑ SUM, MAX, MIN, and AVG Functions

- ■ Can be applied to a set or multiset of numeric values
- ■ Return *the sum, maximum value, minimum value, and average* of those values

Query 19. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

```
Q19:  SELECT  SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
      FROM    EMPLOYEE;
```

Query 20. Find the sum of the salaries of all employees of the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
Q20:  SELECT  SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
      FROM    (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
      WHERE   Dname = 'Research';
```

It returns a single-row summary of all the rows in the EMPLOYEE table

50

SUM, MAX, MIN, AVG

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary) FROM EMPLOYEE;
```

- EMPLOYEE 모든 행의 **단일 행 요약**

- Research 부서만:

```
… FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber) WHERE Dname = 'Research';
```

필기 ✍️

✏ 예.풀이 전체: SUM 281000, MAX 55000, MIN 25000, AVG 35125

Aggregate Functions in SQL (cont.)

❑ COUNT Function

■ Counting *the number of tuples* as specified in a query

Queries 21 and 22. Retrieve the total number of employees in the company (Q21) and the number of employees in the 'Research' department (Q22).

```
Q21:  SELECT   COUNT (*)
      FROM     EMPLOYEE;

Q22:  SELECT   COUNT (*)
      FROM     EMPLOYEE, DEPARTMENT
      WHERE    DNO = DNUMBER AND DNAME = 'Research';
```

The asterisk (*) refers to the rows (tuples),
COUNT (*) returns the number of rows in the result of the query.

■ Counting

■ as specifie

Query 23. Count the number of distinct salary values in the database.

```
Q23:  SELECT   COUNT (DISTINCT Salary)
      FROM     EMPLOYEE;
```

Q1. What if we use COUNT(Salary)?

Q2. NULL values are counted?

51

COUNT

```
SELECT COUNT(*) FROM EMPLOYEE;  – 행(튜플) 수
SELECT COUNT(*) FROM EMPLOYEE, DEPARTMENT WHERE DNO = DNUMBER AND DNAME = 'Research';
SELECT COUNT(DISTINCT Salary) FROM EMPLOYEE;
```

- Q1. COUNT(Salary)면? — 중복 포함 개수
- Q2. NULL도 세나? — **COUNT(속성)은 NULL을 세지 않는다**

필기 ✍️

★ 시험 COUNT(*) = 행 수(NULL 행도) /
COUNT(A) = A가 NULL 아닌 수 /
COUNT(DISTINCT A) = 서로 다른 값 수

Aggregate Functions in SQL (cont.)

❑ **Other Usage of Aggregate Functions**

■ Can be used in selection conditions involving nested queries as well

❑ (1) Specify a nested query with an aggregate function

❑ (2) Use the nested query result in the WHERE clause of an outer query

Q. Retrieve the names of all employees who have two or more dependents

```
SELECT   Lname, Fname
FROM     EMPLOYEE
WHERE    ( SELECT   COUNT (*)
           FROM     DEPENDENT
           WHERE    Ssn = Essn ) >= 2;
```

52

집계 함수의 다른 쓰임

- 중첩 질의 조건에도 쓸 수 있다
  - (1) 집계가 있는 중첩 질의를 만들고
  - (2) 그 결과를 바깥 WHERE에서 비교

```
SELECT Lname, Fname FROM EMPLOYEE WHERE (SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn)
>= 2;
```

  - → 부양가족이 2명 이상인 직원

필기 ✍️

— 넘어가도 되는 쪽

Grouping in SQL

❑ What is Grouping?

- ■ Partition non-overlapping subsets (**groups**) of tuples in a relation
  - ❑ We may want to apply the aggregate functions to *groups of tuples in a relation*
    - ■ E.g., Find the average salary of employees *in each department* or the number of employees who work *on each project*
- ■ Useful when we want to summarize those groups independently

❑ Grouping Attributes

- ■ Attributes used as the criteria for grouping
  - ❑ Each group consists of the tuples that have *the same value for grouping attribute(s)*
- ■ Specified by GROUP BY clause

Query 24. For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
Q24:  SELECT    Dno, COUNT (*), AVG (Salary)
      FROM      EMPLOYEE
      GROUP BY  Dno;
```

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Borg	888665555		55000	NULL	1

Grouping EMPLOYEE tuples by the value of Dno

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

The EMPLOYEE tuples are partitioned into groups,
Each group has the same group attribute value (Dno)

53

그룹화 (Grouping)

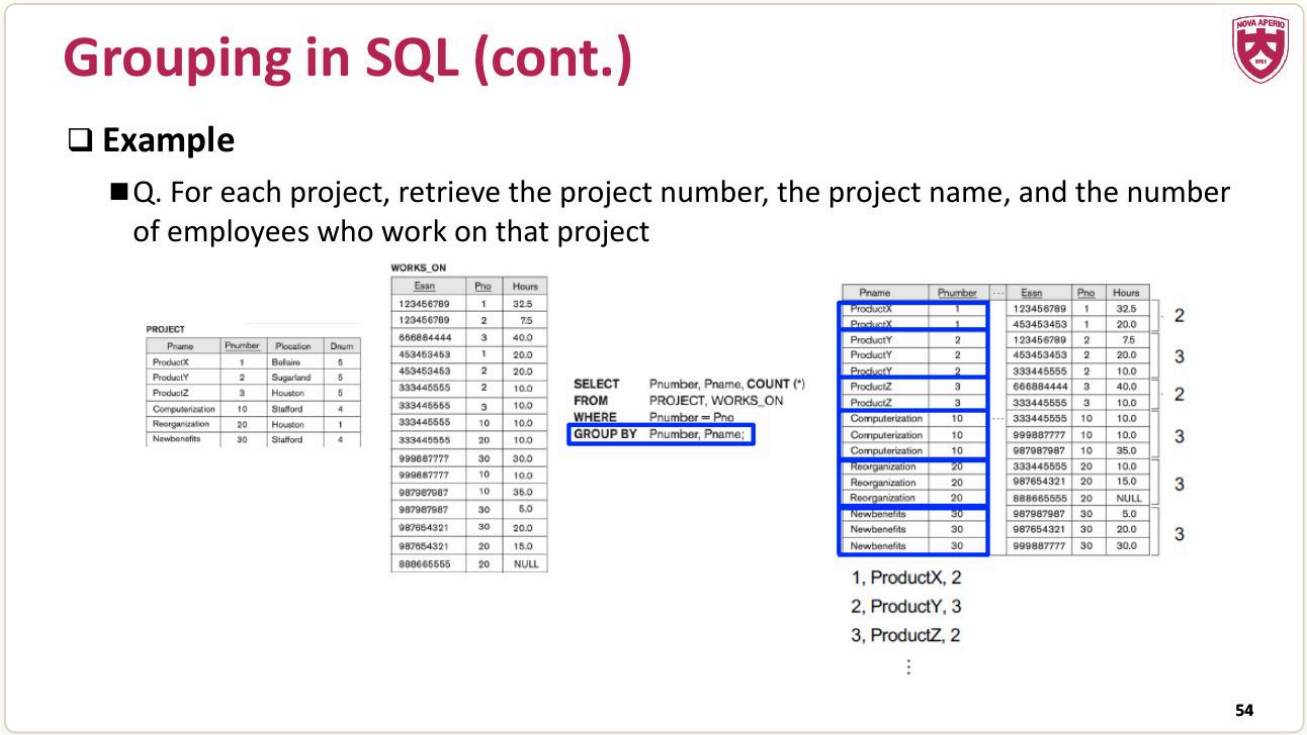
- 튜플을 **겹치지 않는 부분집합(그룹)**으로 나눔
  - 그룹마다 집계 적용 — 부서별 평균 월급, 프로젝트별 인원 등
- **Grouping attributes**: 그룹의 기준 속성 — 같은 값끼리 한 그룹
  - **GROUP BY** 절로 지정

```
SELECT Dno, COUNT(*), AVG(Salary) FROM EMPLOYEE GROUP BY Dno;
```

  - 결과: (5, 4, 33250), (4, 3, 31000), (1, 1, 55000)

필기 ✍️

⚠ 주의 SELECT에 쓴 일반 속성은 GROUP BY에도 있어야 한다



그룹화 예

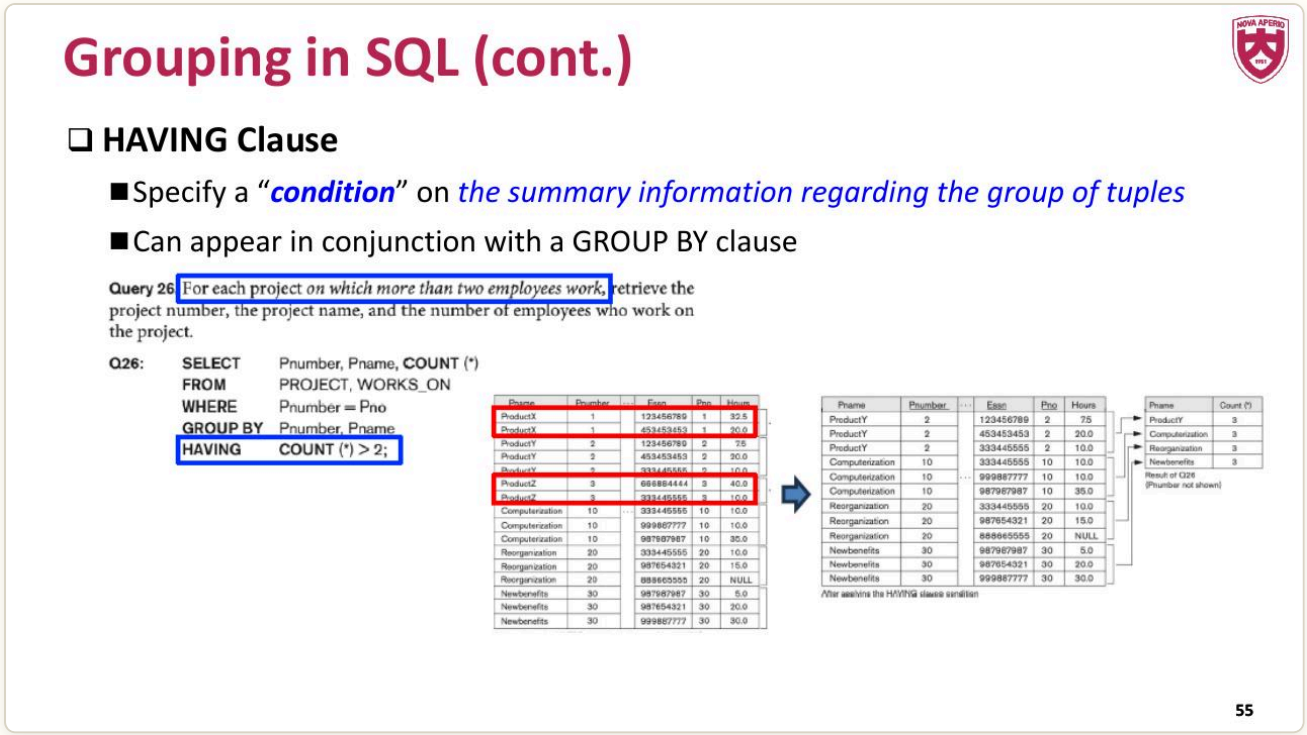
- Q. 프로젝트별 번호, 이름, 일하는 직원 수

```
SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON WHERE Pnumber = Pno GROUP BY
Pnumber, Pname;
```

  - 조인한 뒤 그룹화

필기 ✍️

✏ 예.풀이 1 ProductX 2명, 2 ProductY 3명, 3 ProductZ 2명 ...



HAVING 절

- 그룹의 **요약 정보에 대한 조건**
  - GROUP BY와 함께
- 예(Query 26): 직원이 **2명보다 많은** 프로젝트

```
SELECT Pnumber, Pname, COUNT(*) FROM PROJECT, WORKS_ON WHERE Pnumber = Pno GROUP BY
Pnumber, Pname HAVING COUNT(*) > 2;
```

필기 ✍️

★ 시험 WHERE = 행 거르기(그룹 전) / HAVING = 그룹 거르기(그룹 후)

✏ 예.풀이 결과: ProductY, computerization, Reorganization, Newbenefits (각 3명)

Exercise: Aggregate Functions

❑ Exercise 5: Specify the following query in SQL

■ Q. For each department whose average employee salary is more than $30,000, retrieve the department name and the number of employees working for that department

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Fname	Minit	Lname	Ssn	···	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	···	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Borg	888665555		55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

56

연습 5: 집계 함수

- 평균 월급이 **$30,000보다 많은** 부서마다 부서 이름과 직원 수

필기 ✍️

✏ 예.풀이 정답 p.71 — GROUP BY + HAVING

AVG(Salary) > 30000

Specifying General Constraints & Actions

❑ **Assertions**

■ Specify general constraints that do not fall into any of the built-in constraints

❑ Key constraints, entity and referential integrity constraints

❑ E.g., *the salary of an employee must not be greater than the salary of the manager of the department*

❑ **Triggers**

■ Specify automatic actions to be taken when certain events and conditions occur

❑ E.g., a manager may want to be informed *if an employee's travel expenses exceed a certain limit*



Assertions



Triggers

57

일반 제약과 동작 지정

- **Assertion**: 기본 제약(키, 개체·참조 무결성)에 들지 않는 **일반 제약**
  - 예: 직원 월급은 그 부서 관리자 월급보다 크면 안 된다
- **Trigger**: 특정 사건·조건이 생기면 **자동으로 할 동작**
  - 예: 직원 출장비가 한도를 넘으면 관리자에게 알림

필기 ✍️

★ 시험 Assertion = 제약(막기) / Trigger = 동작(반응하기)

Specifying Constraints as Assertions



❑ The CREATE ASSERTION Command

- ■ Specifying general constraints that users want to check
- ■ Example: SALARY_CONSTRAINT
  - ❑ **CHECK** is followed by a ***condition*** in parentheses() that must hold true on every database state for the assertion to be satisfied
    - ■ Many constraints are usually specified using the **EXISTS** and **NOT EXISTS** style of SQL conditions

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT     *
                     FROM       EMPLOYEE E, EMPLOYEE M,
                                DEPARTMENT D
                     WHERE      E.Salary>M.Salary
                     AND        E.Dno = D.Dnumber
                     AND        D.Mgr_ssn = M.Ssn ) );
```

→ **True**: the result is empty
→ **False**: the result is not empty

Specifying *a query that selects any tuples that violate the desired condition* with a **NOT EXISTS** clause

58

Assertion으로 제약 지정

- **CREATE ASSERTION** — 확인하고 싶은 일반 제약

```
CREATE ASSERTION SALARY_CONSTRAINT CHECK ( NOT EXISTS ( SELECT * FROM EMPLOYEE E, EMPLOYEE
M, DEPARTMENT D WHERE E.Salary > M.Salary AND E.Dno = D.Dnumber AND D.Mgr_ssn = M.Ssn ) );
```

  - CHECK 뒤 괄호 안 조건은 **모든 DB 상태에서 참**이어야
  - 보통 **EXISTS / NOT EXISTS** 스타일로 씀

필기 ✍️

💡 이해 만드는 법: '어기는 튜플을 찾는 질의'를 NOT EXISTS로 감싼다 ⟶ 비면 참

Specifying Actions as Triggers

❑ The CREATE TRIGGER Command

- ■ Specify the type of action to be taken when certain conditions are satisfied
- ■ Example: SALARY_VIOLATION
  - ❑ We want to check whenever an employee's salary is greater than the salary of his/her direct supervisor
  - ❑ Several events can trigger this rule
    - ■ Inserting a new employee tuple, changing an employee's salary, or changing an employee's supervisor
  - ❑ The action to take is to call an external stored procedure SALARY_VIOLATION
- ■ A trigger is regarded as an **ECA (Event, Condition, Action) rule**

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

59

Trigger로 동작 지정

- **CREATE TRIGGER** — 조건이 맞을 때 할 동작
- 예: SALARY_VIOLATION — 직원 월급이 직속 상사보다 크면
  - 촉발 사건 여럿: **새 직원 삽입, 월급 변경, 상사 변경**
  - 동작: 외부 저장 프로시저 호출

```
CREATE TRIGGER SALARY_VIOLATION BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN ON
EMPLOYEE FOR EACH ROW WHEN (NEW.SALARY > (SELECT SALARY FROM EMPLOYEE WHERE SSN =
NEW.SUPERVISOR_SSN)) INFORM_SUPERVISOR(NEW.Supervisor_ssn, NEW.Ssn);
```

- 트리거 = **ECA(Event, Condition, Action) 규칙**

필기 ✍️

★ 시험 ECA = Event · condition · Action (영어 그대로)

Specifying Actions as Triggers: Event

❑ **Three Components of Triggers: Event**

- ■ Update operations explicitly applied to the database
  - ❑ E.g, Inserting a new employee tuple, changing an employee's salary, or changing an employee's supervisor
- ■ All possible events are considered
  - ❑ It may be necessary to write more than one trigger to cover all possible cases
- ■ The events are specified with the keyword **BEFORE** or **AFTER**
  - ❑ The trigger should be executed **before/after** the triggering operation is executed

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

60

트리거 구성 ① Event

- DB에 명시적으로 적용된 **갱신 연산**
  - 예: 새 직원 삽입, 월급 변경, 상사 변경
  - 가능한 모든 사건을 고려 — 여러 트리거가 필요할 수도
  - **BEFORE / AFTER**로 촉발 연산 전후 지정

필기 ✍️

— 넘어가도 되는 쪽

Specifying Actions as Triggers: Condition

❑ **Three Components of Triggers: Condition**

- ■ Determines whether the action should be executed when a triggering event occurs
  - ❑ Once the triggering event has occurred, an *optional condition* may be evaluated
- ■ The condition is specified in **WHEN clause** of the trigger
  - ❑ Then, only if it evaluates to true, the action will be executed
  - ❑ If no condition is specified, the action will be executed once the event occurs

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

61

트리거 구성 ② Condition

- 사건이 생겼을 때 동작을 할지 정함
  - 선택적 — **WHEN** 절
  - 참일 때만 동작 실행
  - 조건이 없으면 사건만으로 실행

필기 ✍️

— 넘어가도 되는 쪽

Specifying Actions as Triggers: Action



❑ **Three Components of Triggers: Action**

- The rule action to be executed automatically
- The action is a sequence of SQL statements, or a database transaction, or an external program

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

Triggers can be used in various applications, such as (1) maintaining database consistency, (2) monitoring database updates, and (3) updating derived data automatically

62

트리거 구성 ③ Action

- 자동으로 실행할 규칙 동작
  - SQL 문의 나열, DB 트랜잭션, 또는 외부 프로그램

필기 ✍️

— 넘어가도 되는 쪽



Q&A

- Basic and Advanced SQL — 끝
- 뒤쪽 p.64~71은 연습문제 정답

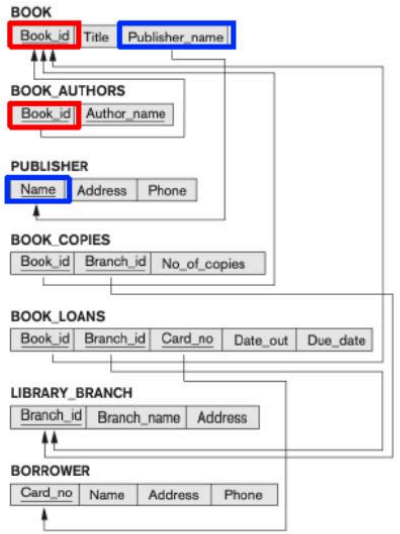
필기 ✍️

— 넘어가도 되는 쪽

Exercises: Specifying Integrity Constraints

❑ **Exercise 1**

- BOOK_AUTHORS.Book_id → BOOK.Book_id
  - ❑ **ON DELETE** CASCADE
    - Once a book is deleted from a database, the information about the authors of the book to be deleted is no longer necessary
  - ❑ **ON UPDATE** CASCADE
    - If the primary key of a book tuple is updated, it should be cascaded to its authors to identify the authors of the book
- BOOK.Publisher_name → PUBLISHER.Name
  - ❑ **ON DELETE** REJECT
    - If there exist books published by a publisher, we do not allow the publisher to be deleted (a book must have a name of the publisher)
  - ❑ **ON UPDATE** CASCADE
    - If the name of a publisher is changed, we want to reflect the change in referencing BOOK tuples



64

연습 1 정답 ① (LIBRARY)

- **BOOK_AUTHORS.Book_id → BOOK**
  - ON DELETE **CASCADE** — 책이 지워지면 그 책의 저자 정보는 필요 없음
  - ON UPDATE **CASCADE** — 책 PK가 바뀌면 저자 쪽도 따라 바뀌어야
- **BOOK.Publisher_name → PUBLISHER.Name**
  - ON DELETE **REJECT** — 출판한 책이 있으면 출판사 삭제 불가
  - ON UPDATE **CASCADE** — 출판사 이름이 바뀌면 책에도 반영

필기 ✍️

★ 시험 ON UPDATE는 거의 다 CASCADE. 삭제는 '남아야 할 정보인가'로 판단

Exercises: Specifying Integrity Constraints

❑ **Exercise 1**

■ BOOK_COPIES.Book_id → BOOK.Book_id

❑ **ON DELETE** CASCADE

- Imagine that a library administrator want to remove a book from the list of all the books in the library
- Then, the administrator wants to delete the corresponding tuples in BOOK
- Thus, any copies of this book do not need to be maintained anymore

❑ **ON UPDATE** CASCADE

- If there is a change in Book_id of a BOOK tuple, the change should be reflected into the referencing tuples (i.e., its copies)

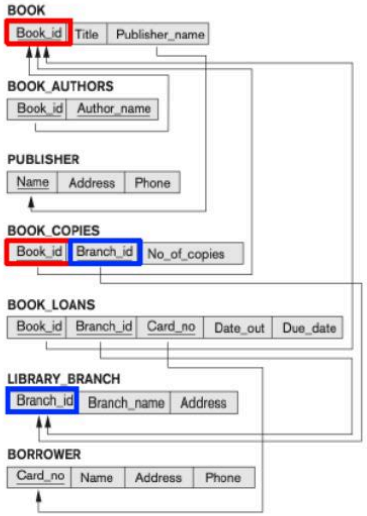
■ BOOK_COPIES.Branch_id → LIBRARY_BRANCH.Branch_id

❑ **ON DELETE** CASCADE

- The deletion of a LIBRARY_BRANCH tuple: actual "closing" of that branch
- Thus, the book copies do not need to be kept track of anymore

❑ **ON UPDATE** CASCADE

- If there is a change in Branch_id, the change should be reflected into the referencing tuples to correctly keep track of the library branch where the book copies are



65

연습 1 정답 ②

- **BOOK_COPIES.Book_id → BOOK**
  - ON DELETE **CASCADE** — 목록에서 책을 빼면 사본도 관리할 필요 없음
  - ON UPDATE **CASCADE**
- **BOOK_COPIES.Branch_id → LIBRARY_BRANCH**
  - ON DELETE **CASCADE** — 지점 삭제 = 실제 폐쇄, 사본 추적 불필요
  - ON UPDATE **CASCADE**

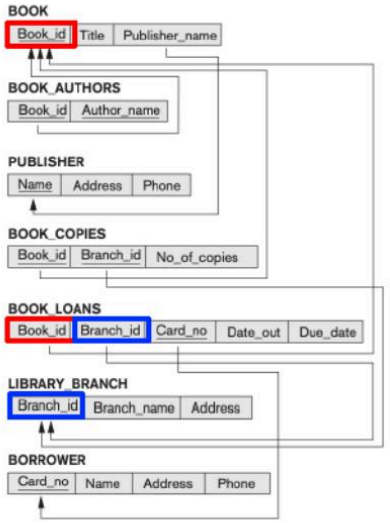
필기 ✍️

— 넘어가도 되는 쪽

Exercises: Specifying Integrity Constraints

❑ **Exercise 1**

- BOOK_LOANS.Book_id → BOOK.Book_id
  - ❑ **ON DELETE** REJECT (or CASCADE)
    - Imagine if a library administrator deletes a BOOK tuple since the book is removed from the list of all the books in the library
    - However, it does not mean that there are no any loans left, and the library want all books to be returned someday
  - ❑ **ON UPDATE** CASCADE
    - If there is a change in Book_id, the change should be reflected into the referencing tuples in order to correctly keep track of all the book loans
- BOOK_LOANS.Branch_id → LIBRARY_BRANCH.Branch_id
  - ❑ **ON DELETE** CASCADE (or REJECT)
    - The deletion of a LIBRARY_BRANCH tuple: actual "closing" of that branch
    - Thus, the loaned books do not need to be returned (forgive all the loans…)
  - ❑ **ON UPDATE** CASCADE
    - If there is a change in Branch_id, the change should be reflected into the referencing tuples to correctly keep track of the library branch where the book is loaned



66

연습 1 정답 ③

- **BOOK_LOANS.Book_id → BOOK**
  - ON DELETE **REJECT (또는 CASCADE)** — 목록에서 빼도 대출 중인 책은 돌려받아야
  - ON UPDATE **CASCADE**
- **BOOK_LOANS.Branch_id → LIBRARY_BRANCH**
  - ON DELETE **CASCADE (또는 REJECT)** — 지점 폐쇄면 대출을 면제
  - ON UPDATE **CASCADE**

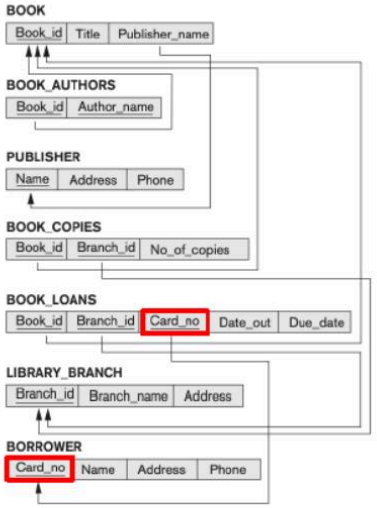
필기 ✍️

⚠ 주의 괄호 속 다른 답도 인정 — 근거를 같이 써야 함

Exercises: Specifying Integrity Constraints

❑ Exercise 1

- BOOK_LOANS.Card_no → BORROWER.Card_no
  - ❑ **ON DELETE** REJECT
    - A library policy do not often permit a member (borrower) to leave the library until the member returns all the loaned books (or paying for them)
    - Thus, the deletion of a referenced tuple (BORROWER) is rejected
  - ❑ **ON UPDATE** CASCADE
    - If there is a change in BORROWER.Card_no, the change should be reflected into the referencing tuples (book loans) in order to correctly keep track of the certain borrower who has loaned the book



67

연습 1 정답 ④

- **BOOK_LOANS.Card_no → BORROWER.Card_no**
  - ON DELETE **REJECT** — 빌린 책을 다 돌려주거나 변상할 때까지 탈퇴 불가
  - ON UPDATE **CASCADE** — 카드 번호가 바뀌면 대출 기록에도 반영

필기 ✍️

— 넘어가도 되는 쪽

Exercises: Data Retrieval in SQL

❑ Exercise 2: Specify the Following Queries in SQL

■ Q. Retrieve the names of all employees in department 5 who work more than 10 hours per week on the "ProductX" project

Projection attributes

```
SELECT E.Fname, E.Minit, E.Lname
FROM EMPLOYEE AS E, PROJECT AS P, WORKS_ON AS W
WHERE E.Dno=5 AND P.Pname='ProductX' AND W.Hours > 10
AND E.Ssn=W.Essn AND E.Dno=P.Dnum;
```

3 Selection and 2 Join conditions

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jebbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

68

연습 2 정답

```
SELECT E.Fname, E.Minit, E.Lname FROM EMPLOYEE AS E, PROJECT AS P, WORKS_ON AS W WHERE
E.Dno = 5 AND P.Pname = 'ProductX' AND W.Hours > 10 AND E.Ssn = W.Essn AND E.Dno = P.Dnum;
```

- 선택 조건 3개 + 조인 조건 2개

필기 ✍️

⚠ 주의 슬라이드는 마지막 조인을 E.Dno = P.Dnum으로 씀 → WORKS_ON과 PROJECT가 안 이어져서 'ProductX에서' 10시간이 보장 안 됨. W.Pno = P.Pnumber가 맞다 (시험엔 이쪽으로)

Exercises: Data Retrieval in SQL

❑ Exercise 3: Specify the Following Queries in SQL

■ Q. Retrieve the name and transcript of each senior student (Class=4) majoring in CS. A transcript includes 1) course name, 2) course number, 3) credit hours, 4) semester, 5) year, and 6) grade for each course completed by the student

```
SELECT   S1.Name, C.Course_name, C.Course_number, C.Credit_hours, S2.Semester, S2.Year, G.Grade
FROM     STUDENT AS S1, COURSE AS C, SECTION AS S2, GRADE_REPORT AS G
WHERE    S1.Class = '4' AND S1.Major = 'CS' AND S1.Student_number = G.Student_number
         AND G.Section_identifier = S2.Section_identifier AND S2.Course_number = C.Course_number;
```

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

69

연습 3 정답

```
SELECT S1.Name, C.Course_name, C.Course_number, C.Credit_hours, S2.Semester, S2.Year,
G.Grade FROM STUDENT AS S1, COURSE AS C, SECTION AS S2, GRADE_REPORT AS G WHERE S1.Class =
'4' AND S1.Major = 'CS' AND S1.Student_number = G.Student_number AND G.Section_identifier =
S2.Section_identifier AND S2.Course_number = C.Course_number;
```

필기 ✍️

💡 이해 표 4개 ⟶ 조인 조건 3개 (표 n개면 조인 조건 n−1개)

Exercise: Boolean Functions

❑ Exercise 4: Specify the Following Query

■ Q. Retrieve the name of each employee who works on all the projects controlled by department number 5

```
SELECT    Fname, Lname
FROM      EMPLOYEE
WHERE     NOT EXISTS (  SELECT    Pnumber
                        FROM      PROJECT
                        WHERE     Dnum = 5)
                        EXCEPT  ( SELECT    Pno
                                  FROM      WORKS_ON
                                  WHERE     Ssn = Essn) );
```

Set difference operation:
Pnumber and Pno are compatible

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

70

연습 4 정답

```
SELECT Fname, Lname FROM EMPLOYEE WHERE NOT EXISTS ( (SELECT Pnumber FROM PROJECT WHERE
Dnum = 5) EXCEPT (SELECT Pno FROM WORKS_ON WHERE Ssn = Essn) );
```

- 5번 부서 프로젝트 중 '내가 안 하는 것'이 하나도 없으면 → 전부 한다

필기 ✍️

★ 시험 '모든' = NOT EXISTS (전체 EXCEPT 내 것) — 외워 두기

Exercise: Aggregate Functions



❑ Exercise 5: Specify the following query in SQL

■ Q. For each department whose average employee salary is more than $30,000, retrieve the department name and the number of employees working for that department

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Fname	Minit	Lname	Ssn	···	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	···	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Borg	888665555		55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

```
SELECT      D.Dname, COUNT(*)
FROM        EMPLOYEE AS E, DEPARTMENT AS D
WHERE       E.Dno = D.Dnumber
GROUP BY    E.Dno
HAVING      AVG(E.Salary) > 30000
```

71

연습 5 정답

```
SELECT D.Dname, COUNT(*) FROM EMPLOYEE AS E, DEPARTMENT AS D WHERE E.Dno = D.Dnumber GROUP
BY E.Dno HAVING AVG(E.Salary) > 30000;
```

필기 ✍️

⚠ 주의 교안 표기는 GROUP BY E.Dno — 표준 SQL에선 SELECT의 D.Dname도 GROUP BY에 넣는 게 안전 (GROUP BY D.Dnumber, D.Dname)